



DEPARTMENT OF BCA

DATA ANALYTICS

1

SUBJECT: PYTHON PROGRAMMING
SEMESTER: 4th BCA ITH



PYTHON

PROGRAMMING

PYTHON FOUNDER

"If you decide to design your own language, there are thousands of sort of amateur language designer pitfalls."

Guido van Rossum

MODULE - 1

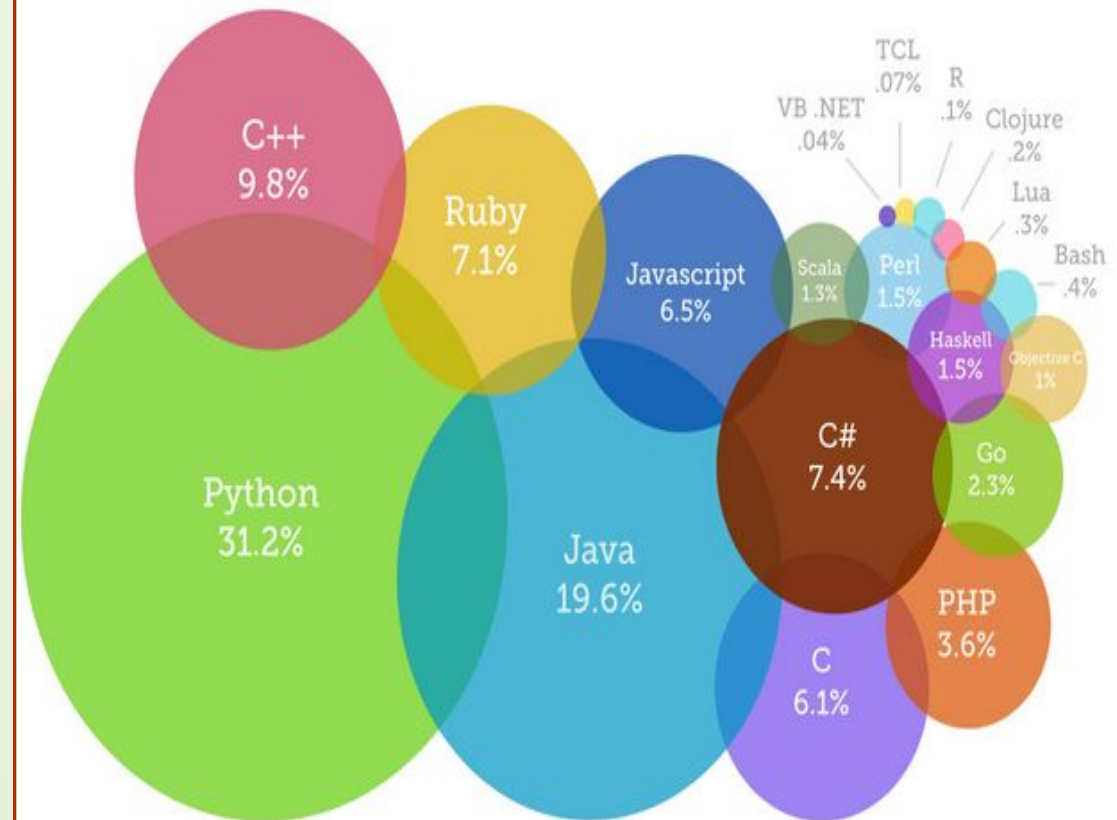
❑ INTRODUCTION TO PYTHON PROGRAMMING

- ❑ Overview-Applications -Print Statement-Writing and Executing a Python Program-Identifiers-Indentations- Keywords-Operators-Control Statements-Strings (Slicing- Indexing- Concatenation- other operations on Strings) Functions

INTRODUCTION TO PYTHON

- Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language.
- It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL).

Most Popular Coding Languages of 2015



- ❑ Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable.
- ❑ It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.
- ❑ Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- ❑ Python is Interactive – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- ❑ Python is Object-Oriented – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- ❑ Python is a Beginner's Language – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

7

- ❑ Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.
- ❑ Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.
- ❑ Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).
- ❑ Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Java vs. Python in 2019

- Java and Python have many similarities. Both languages have strong cross-platform support and extensive standard libraries. They both treat (nearly) everything as objects.
- Both languages compile to bytecode, but Python is (usually) compiled at runtime. They are both members of the Algol family, although Python deviates further from C/C++ than Java does.

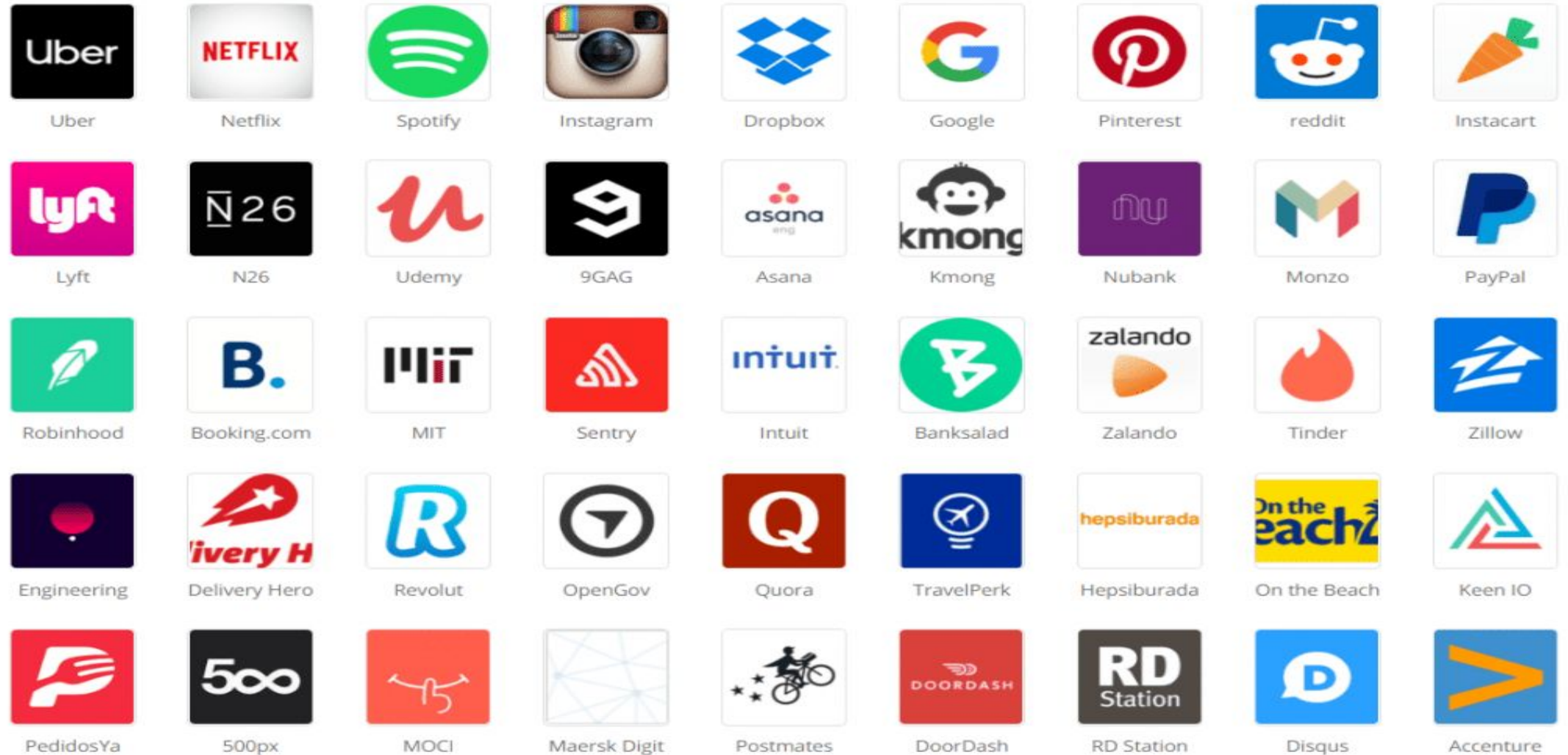
Some of the applications of Python are

- ❑ Web Development (using Django framework)
- ❑ Data Science (Machine Learning – PyBrain, scikit-learn etc)
- ❑ Data Analytics (matplotlib)
- ❑ Artificial Intelligence (AIMA, pyDatalog)
- ❑ Scientific Computing (numpy)
- ❑ Image Processing (OpenCV)
- ❑ Signal Processing (scipy.signal)
- ❑ Robotics (ROS – Robotics OS, pyro)
- ❑ Circuit Simulation (PySpice)
- ❑ Games

Most Popular websites built using Python are

10

6136 companies reportedly use Python in their tech stacks, including Uber, Netflix, and Spotify.



Python IDEs

PyCharm

Visual
Studio Code

Sublime Text



Vim



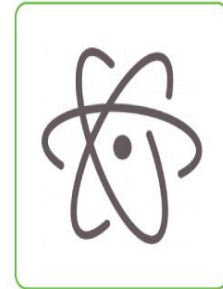
GNU Emacs



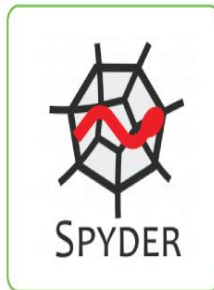
IDLE

Python's
Integrated
Development
and Learning
Environment

Atom



Spyder



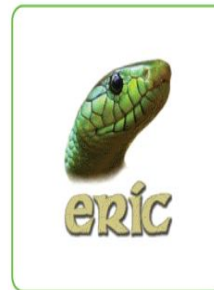
JuPyter



Eclipse



Eric Python



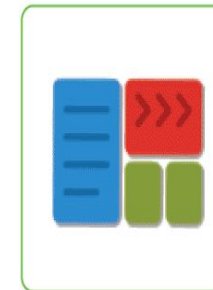
Wing



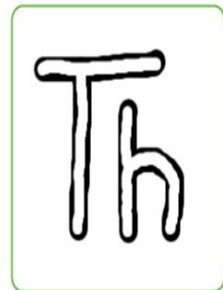
PyScripter



Pyzo



Thonny

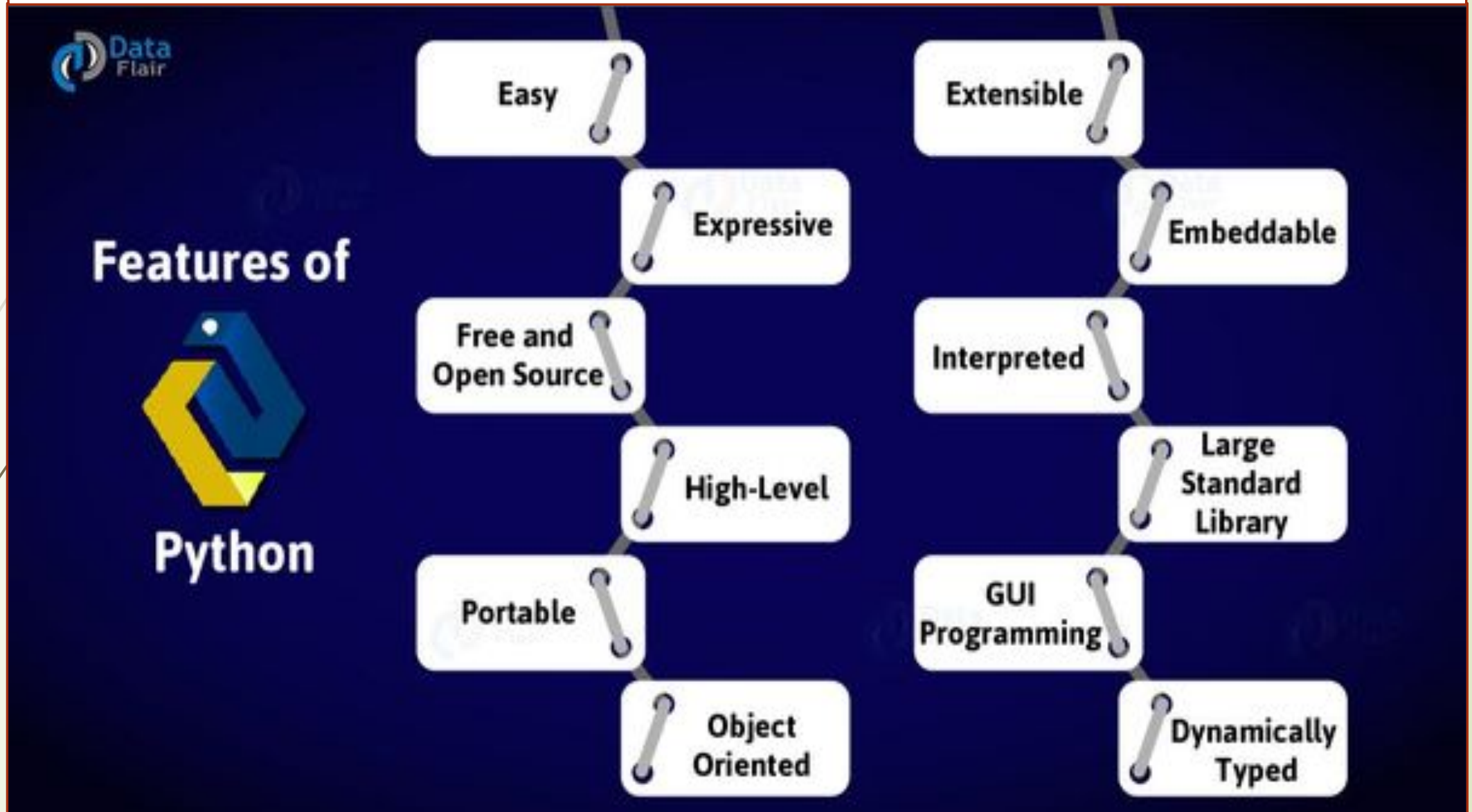


Or use Google colab



Python Features

12



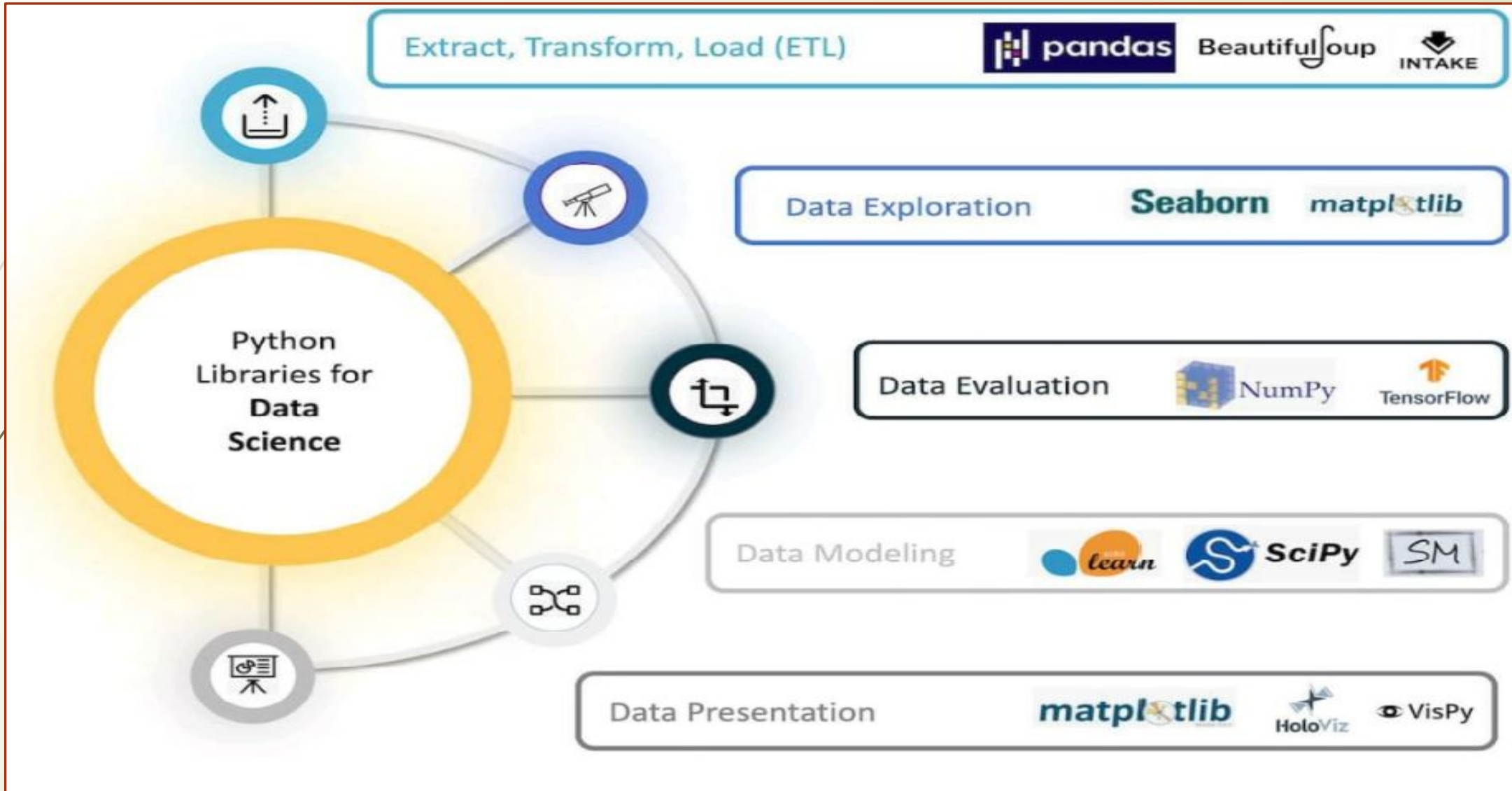
Python Features

- ❑ **Easy-to-learn** – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- ❑ **Easy-to-read** – Python code is more clearly defined and visible to the eyes.
- ❑ **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- ❑ **A broad standard library** – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- ❑ **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- ❑ **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- ❑ **Extendable** – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- ❑ **Databases** – Python provides interfaces to all major commercial databases.
- ❑ **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- ❑ **Scalable** – Python provides a better structure and support for large programs than shell scripting.

Python Installation

- ❑ Goto below link
- ❑ <https://www.anaconda.org/>
- ❑ Download anaconda in right top corner
- ❑ Click on Download for Anconda to launch Jupyter lab/Spyder
- ❑ Then scroll down the same page
- ❑ download Python 3.7 Version for windows(64/32 bit)
- ❑ Install Pthon 3.7 for console

Python Libraries for Data Science

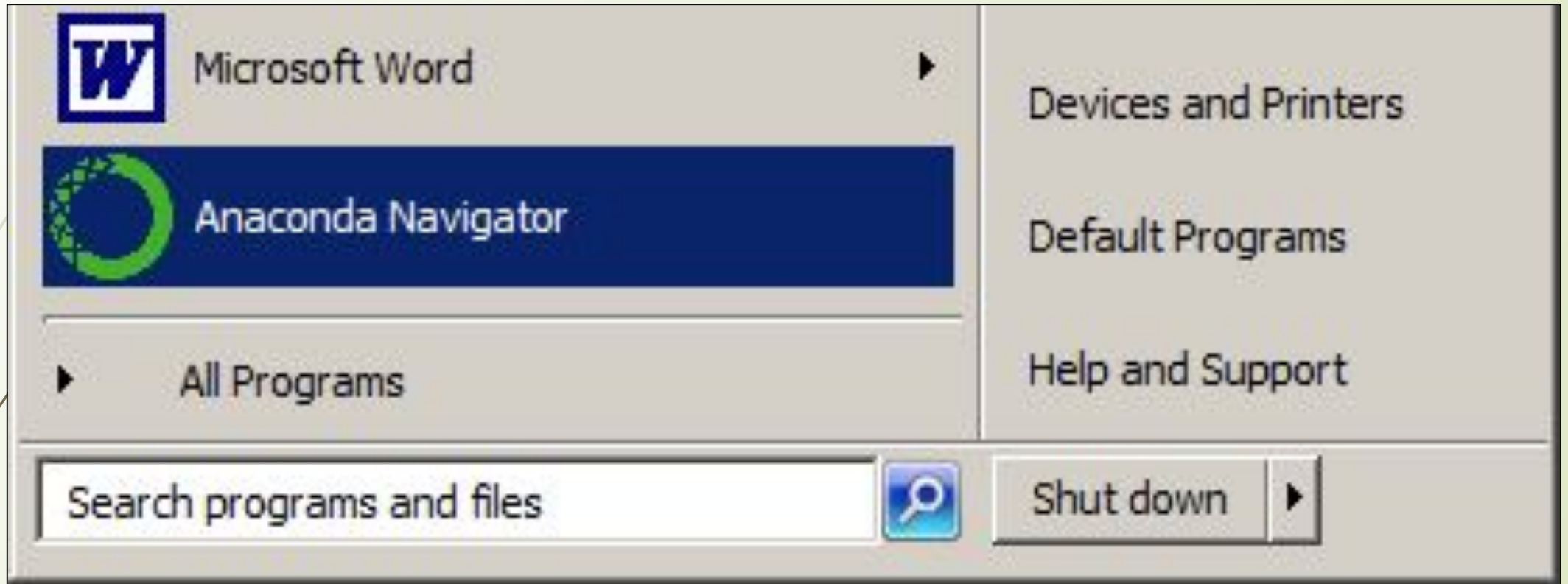


Jupyter

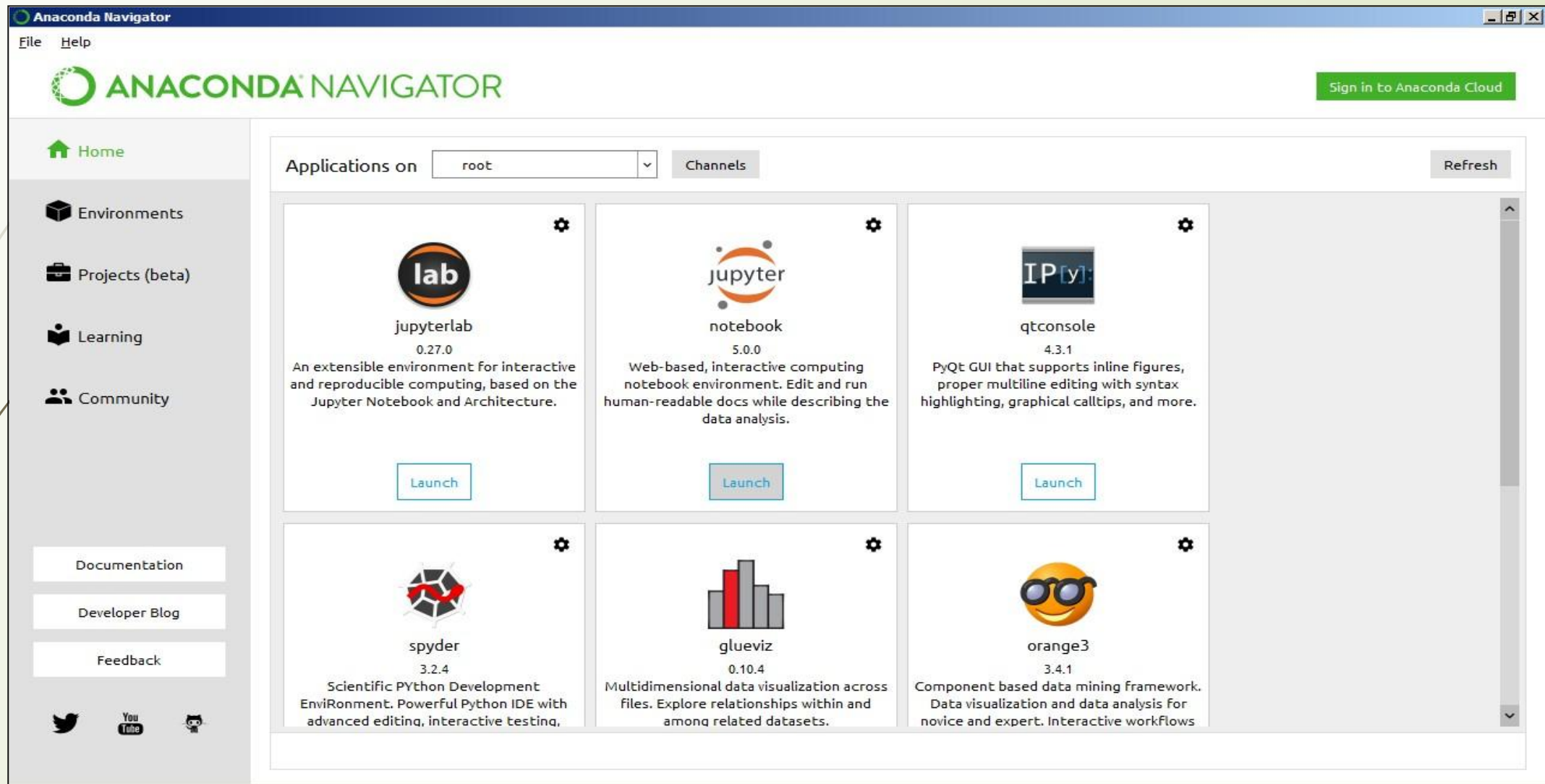
16

- ❑ Jupyter is a loose acronym meaning Julia, Python, and R. These programming languages were the first target languages of the Jupyter application, but nowadays, the notebook technology also supports many other languages.
- ❑ In 2001, Fernando Pérez started developing IPython. **IPython** is a command shell for interactive computing in multiple programming languages, originally developed for the Python.
- ❑ Consider the following features provided by IPython:
 - ❑ Interactive shells (terminal and Qt-based).
 - ❑ A browser-based notebook with support for code, text, mathematical expressions, inline plots and other media.
 - ❑ Support for interactive data visualization and use of GUI toolkits.
 - ❑ Flexible, embeddable interpreters to load into one's own projects.
- ❑ In 2014, Fernando Pérez announced a spin-off project from IPython called Project Jupyter. IPython will continue to exist as a Python shell and a kernel for Jupyter, while the notebook and other language-agnostic parts of IPython will move under the Jupyter name. Jupyter added support for Julia, R, Haskell and Ruby.
- ❑ To start the Jupyter notebook, open Anaconda navigator (a desktop graphical user interface included in Anaconda that allows you to launch applications and easily manage Conda packages, environments and channels without the need to use command line commands).

Anaconda Navigator

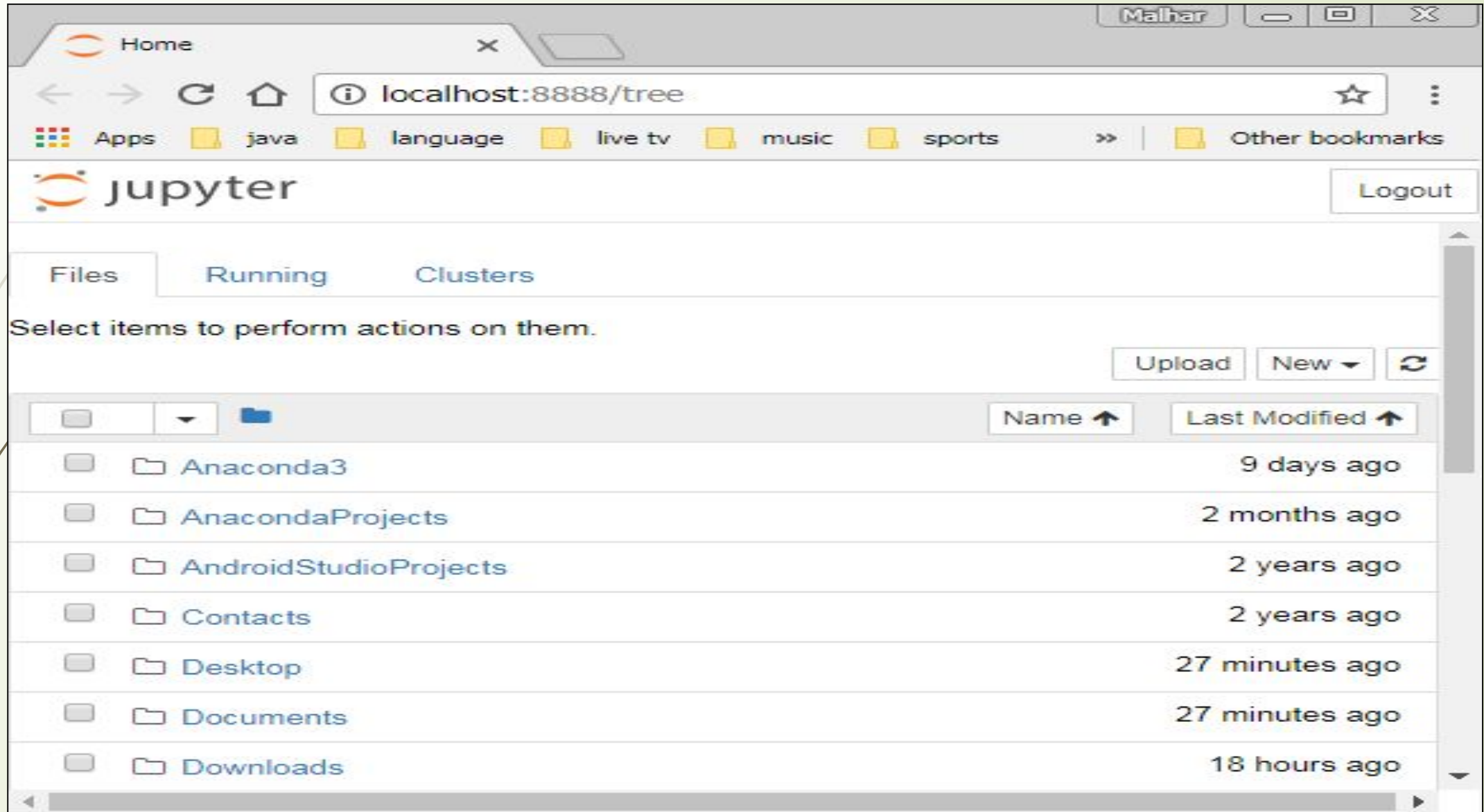


Navigator displays the installed components in the distribution.

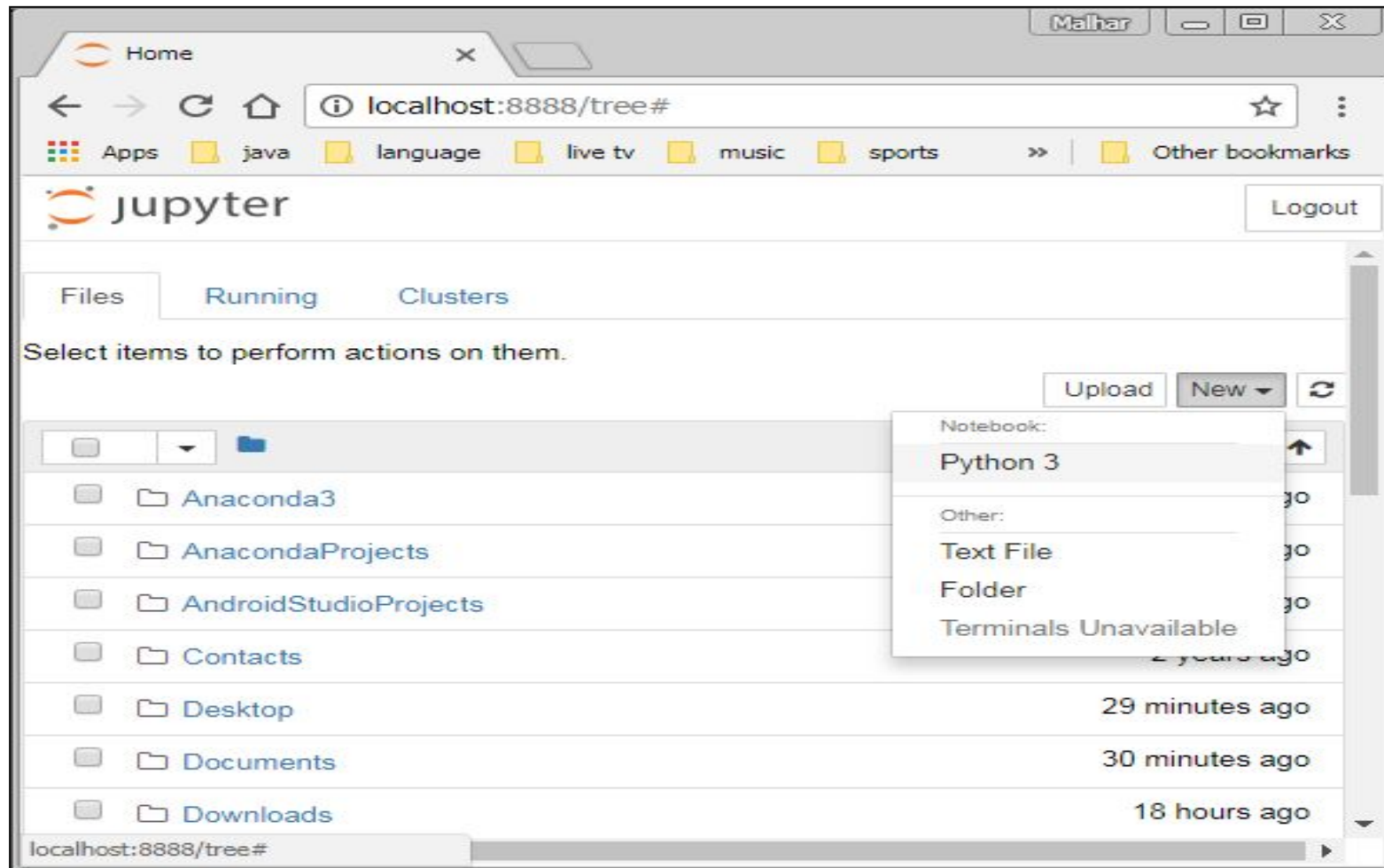


Launch Jupyter Notebook from the Navigator:

19

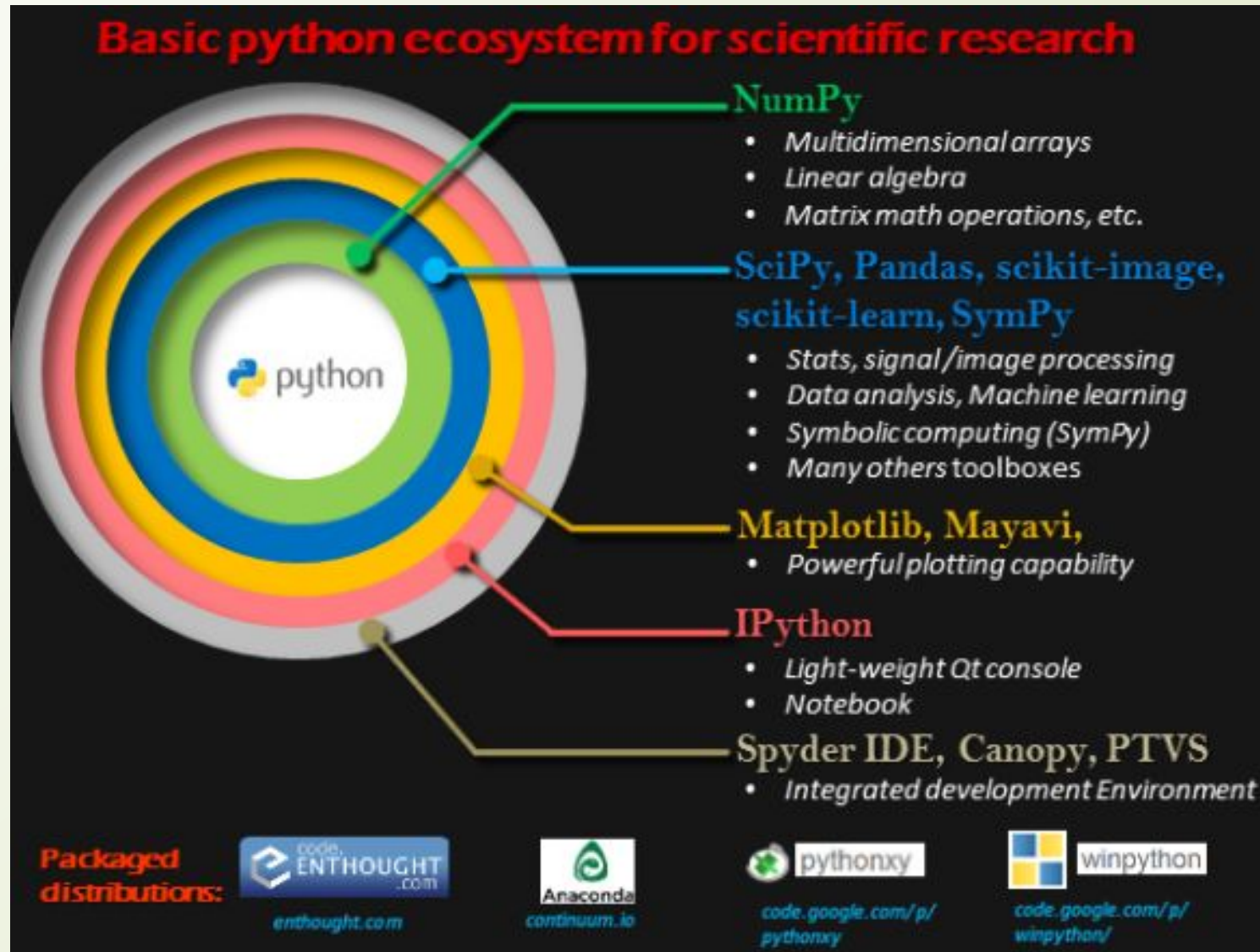


- You probably want to start by making a new notebook. You can easily do this by clicking on the "New button" in the "Files tab". You see that you have the option to make a regular text file, a folder, and a terminal. Lastly, you will also see the option to make a Python 3 notebook.



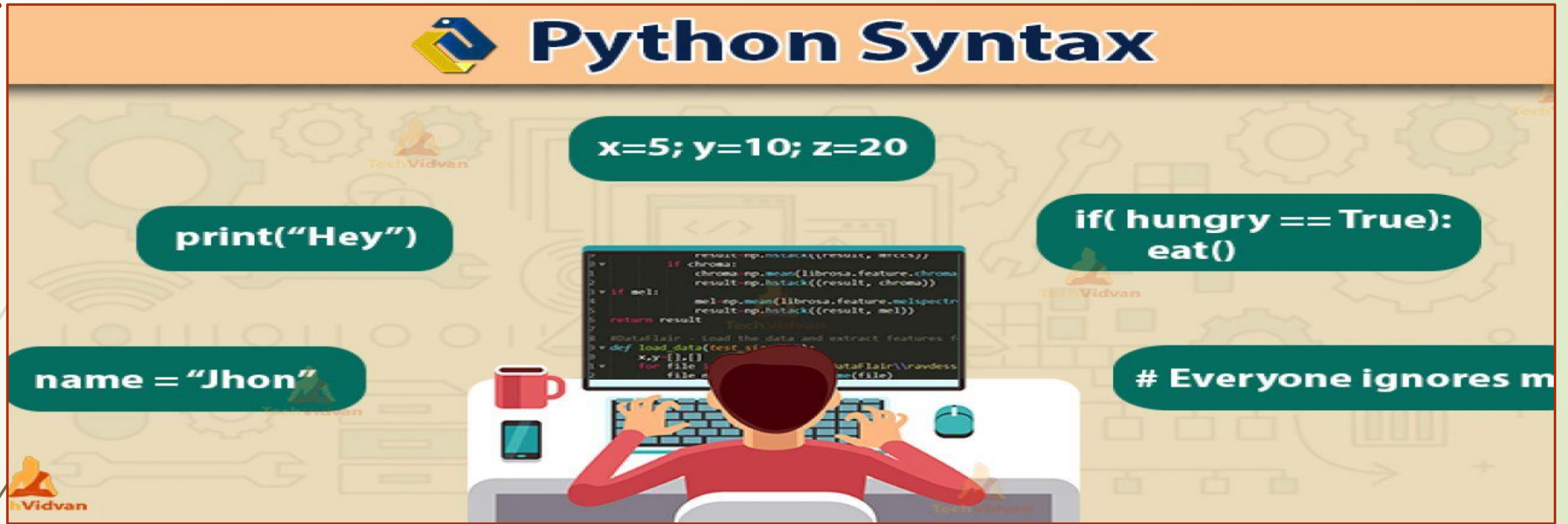
Extensive Library

21



Basic Syntax

22



- ❑ Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.
- ❑ Python files have the extension **.py**. Type the following source code in a test.py file –

print() Function

- ❑ The print() function prints the given object to the standard output device (screen) or to the text stream file.
- ❑ The full syntax of print() is:

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```
- ❑ print() Parameters
- ❑ •**objects** - object to be printed. * indicates that there may be more than one object
- ❑ •**sep** - objects are separated by sep. Default value: ' '
- ❑ •**end** - end is printed at last
- ❑ •**file** - must be an object with write(string) method. If omitted it, sys.stdout will be used which prints objects on the screen.
- ❑ •**flush** - If True, the stream is forcibly flushed. Default value: False

Examples

24

□ Example 1: How print() works in Python?

```
print("Python is fun.")  
a = 5  
# Two objects are passed  
print("a =", a)  
b = a  
# Three objects are passed  
print('a =', a, '= b')
```

Example 2: print() with separator and end parameters

```
a = 5  
print("a =", a, sep='00000', end='\n\n\n')  
print("a =", a, sep='0', end="")
```

3. Print an object without a trailing newline
By default, the print function add a trailing line.

```
>>> x, y, z = 1, 2, 3  
>>> print(x, y); print(y, z)  
1 2  
2 3
```

4. `print('hello', 'world', sep=None)`
`print('hello', 'world', sep="")`
`print('hello', 'world', sep=' * ')`
`print('hello', '\nWelcome to', '\npython')`
`print('hello', 'world', sep='\n')`
`print('hello', 'world', 'welcome', sep='\n')`
`>>> print(1, 'Python Tricks', 'Sen', sep=',')`
1,Python Tricks,Sen

Python Identifiers

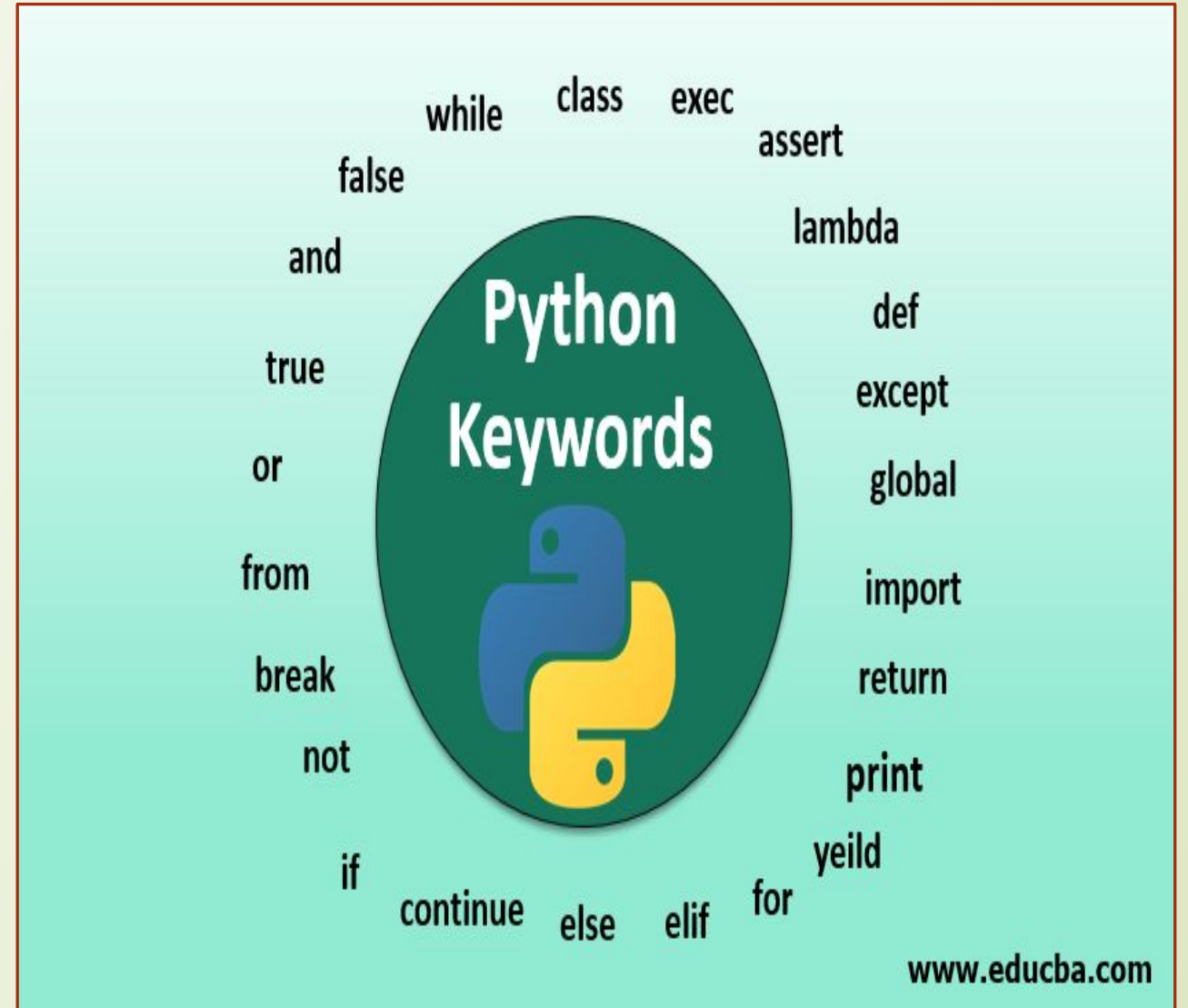
- ❑ A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- ❑ Python does not allow punctuation characters such as @, \$, and % within identifiers. Python is a case sensitive programming language. Thus, **Manpower** and **manpower** are two different identifiers in Python.
- ❑ Here are naming conventions for Python identifiers –
- ❑ Class names start with an uppercase letter. All other identifiers start with a lowercase letter.
- ❑ Starting an identifier with a single leading underscore indicates that the identifier is private.
- ❑ Starting an identifier with two leading underscores indicates a strong private identifier.
- ❑ If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

Example

```
myVariable = 10
def my_function():
    print("Hello, World!")
Class MyClass:
    pass
my_object = MyClass()
print(id(myVariable))
print(id(my_function))
print(id(my_object))
```

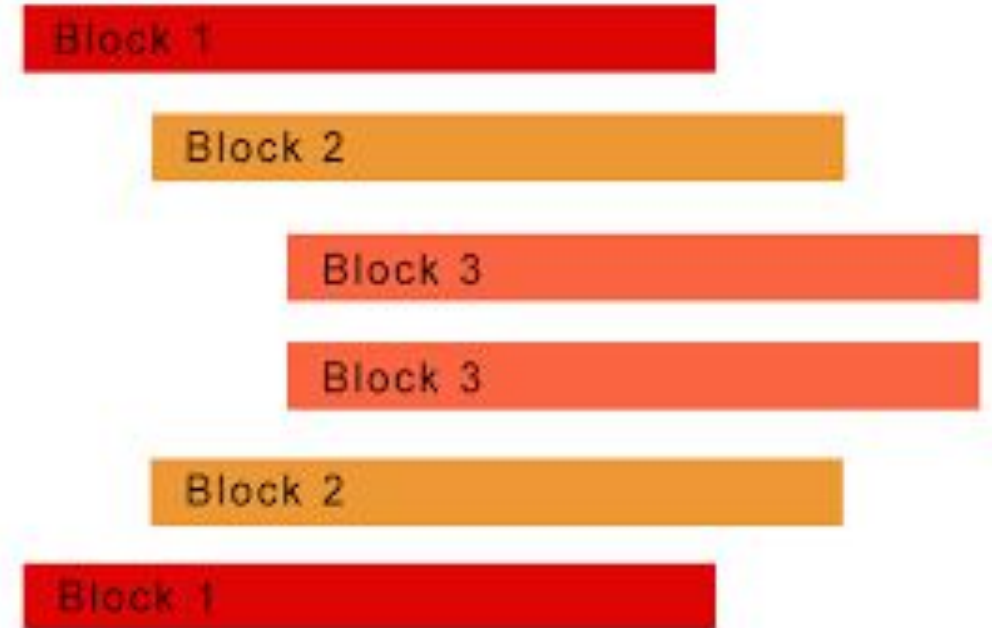
Reserved Words

The list shows the Python keywords. These are reserved words and you cannot use them as constants or variables or any other identifier names. All the Python keywords contain lowercase letters only.



Lines and Indentation

- ❑ Python **does not use braces({})** to indicate blocks of code for class and function definitions or flow control.
- ❑ Blocks of code are denoted by line indentation, which is rigidly enforced.
- ❑ The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount.



```
Statement
if condition:
    if condition:
        Statement
    else:
        Statement
Statement
```

How the interpreter
visualises

```
Code block 1 begins
Code block 1 continues
Code block 2 begins
    Code block 3 begins
    Code block 2 continues
    Code block 3 continues
Code block 1 continues
```

Multi-Line Statements

- ❑ Statements in Python typically end with a new line. Python, however, allows the use of the line **continuation character (\)** to denote that the line should continue.
- ❑ For example –
total = item_one + \
item_two + \
item_three

Quotation in Python

- ❑ Python accepts single ('), double (") and triple (''' or """) quotes to denote string literals, as long as the same type of quote starts and ends the string.
- ❑ The triple quotes are used to span the string across multiple lines. For example, all the following are legal –
 - ❑ `word = 'word'`
 - ❑ `sentence = "This is a sentence."`
 - ❑ **`paragraph = """This is a paragraph. It is`**
 - ❑ **`made up of multiple lines and sentences."""`**
- ❑ **Comments in Python**
 - ❑ A hash sign (#) that is not inside a string literal is the beginning of a comment. All characters after the #, up to the end of the physical line, are part of the comment and the Python interpreter ignores them.

- ❑ `# First comment print ("Hello, Python!") # second comment`
- ❑ This produces the following result
- ❑ Hello, Python!
- ❑ You can type a comment on the same line after a statement or expression –
- ❑ `name = "Sonu" # This is again comment`
- ❑ You can comment multiple lines as follows
- ❑ `# This is a comment.# This is a comment, too.# This is a comment, too.# I said that already.`
- ❑ Following triple-quoted string is also ignored by Python interpreter and can be used as a multiline comments:
- ❑ **`"""This is a multiline comment."""`**

❑ Multiple Statements on a Single Line

- ❑ The semicolon (;) allows multiple statements on a single line given that no statement starts a new code block. Here is a sample snip using the semicolon
import sys; x = 'Green Ray'; print(x)

❑ Multiple Statement Groups as Suites

- ❑ Groups of individual statements, which make a single code block are called **suites** in Python. Compound or complex statements, such as if, while, def, and class require a header line and a suite.
- ❑ Header lines begin the statement (with the keyword) and terminate with a colon (:) and are followed by one or more lines which make up the suite.
- ❑ For example –

```
if expression :  
    suite  
elif expression :  
    suite  
else :  
    suite
```

Variable Types

33

- ❑ Variables are nothing but reserved memory locations to store values.
- ❑ Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to the variables, you can store integers, decimals or characters in these variables.
- ❑ **Assigning Values to Variables**
- ❑ Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.
- ❑ Example:

```
counter = 100      # An integer assignment
miles  = 1000.0    # A floating point
name   = "John"    # A string
print(counter)
print(miles)
print(name)
```

Multiple Assignment

Python allows you to assign a single value to several variables simultaneously.

For example –

```
a = b = c = 1
```

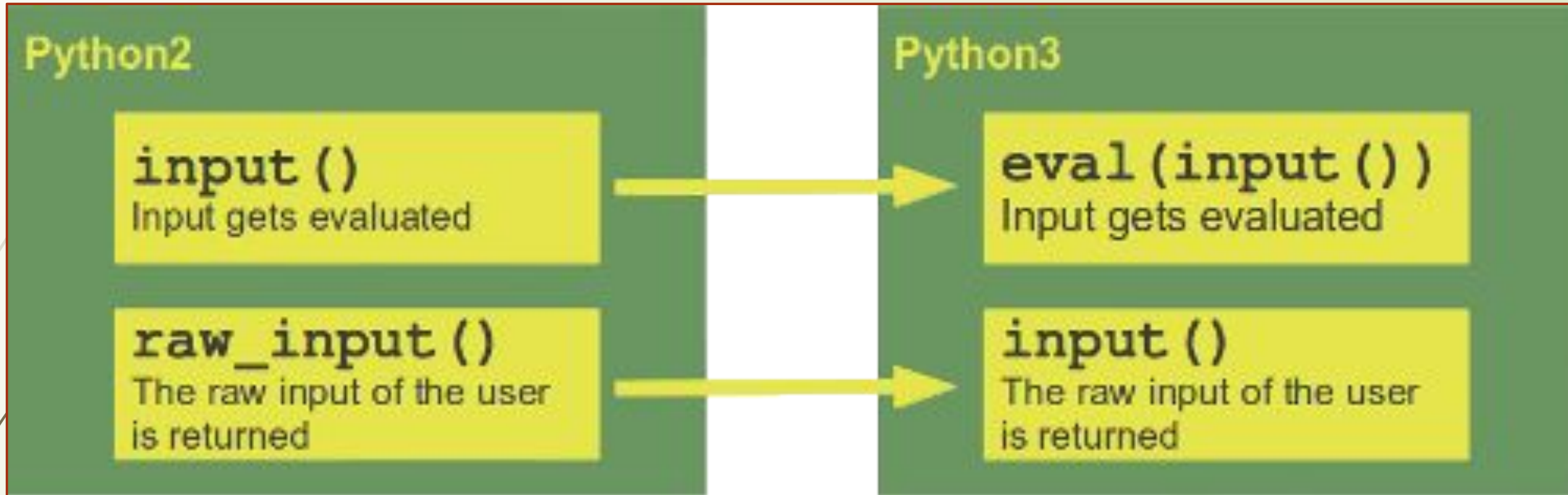
Here, an integer object is created with the value 1, and all the three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

For example

```
a, b, c = 1, 2, "john"
```

Here, two integer objects with values 1 and 2 are assigned to the variables a and b respectively, and one string object with the value "john" is assigned to the variable c.

Input() to read input from end user



Syntax:

Var_name=input()/input("msg")

Examples on Input function

36

```
❑ Example 1# Program to check input
❑ # type in Python
❑ a = input ("Enter number :")
❑ print(a)
❑ f_name = input("Enter name : ")
❑ print(f_name)
❑ # Printing type of input value
❑ print ("type of number", type(a))
❑ print ("type of name", type(f_name))
```



1. Using split() function
2. Using input () function
3. Using map () function
4. Using List Comprehension.

1. split ()

split() function helps us get multiple inputs from the user and assign them to the respective variables in one line. This function is generally used to separate a given string into several substrings. The function generally breaks the given input by the specified separator and in case the separator is not provided then any white space is considered as a separator.

Syntax for using the split function to take multiple inputs is

variable 1, variable 2 = input("Enter what has to be conveyed to the user here"). split()

for space-separated

Input()

```
# Python program to take multiple inputs from the user  
a, b = input("Enter two lucky number: ").split()  
print("First lucky number is: ", a)  
print("Second lucky number is: ", b)
```

Output:

```
Enter two of your lucky number: 7 1  
First lucky number is: 7  
Second lucky number is: 1
```

```
x = int(input("Enter value for X : "))  
y = int(input("Enter value for Y : "))  
print("The sum is ", x+y)
```

Map() function

- ❑ map() function returns a map object(which is an iterator) of the results after applying the given function to each item of a given iterable (list, tuple etc.)
- ❑ Syntax :
- ❑ map(fun, iter)
- ❑ Parameters :
- ❑ fun : It is a function to which map passes each element of given iterable.
- ❑ iter : It is a iterable which is to be mapped.
- ❑ **#example on multiple inputs in Python using map**

```
x, y = map(int, input("Enter two values: ").split())
```

```
print("First Number is: ", x)
```

```
print("Second Number is: ", y)
```

- ❑ **Output:**
- ❑ Enter two values: 7 1
- ❑ First Number is: 7
- ❑ Second Number is: 1

#Example on multiple inputs in Python using map

```
x, y = map(str, input("Enter your first and last name ").split())  
print("First Name is: ", x)  
print("Second Name is: ", y)
```

Python Lambda Function λ

Anonymous Function

Python allows to create anonymous **function** i.e **function** having no names using a facility called **lambda function**. **Lambda functions** are small **functions** usually not more than a line.

keyword ← argument

```
f = lambda a: a * a
```

function object that accepts and stores the result of the expression ← one-line expression

Double all numbers using map and lambda

```
numbers = (1, 2, 3, 4)
result = map(lambda x: x + x, numbers)
print(list(result))
```

- ❑ # Add two lists using map and lambda
- ❑ numbers1 = [1, 2, 3]
- ❑ numbers2 = [4, 5, 6]
- ❑ result = map(lambda x, y: x + y, numbers1, numbers2)
- ❑ print(list(result))
- ❑ Output: 5 7 9

- ❑ # Double all numbers using map and lambda
- ❑ numbers = (1, 2, 3, 4)
- ❑ result = map(lambda x: x + x, numbers)
- ❑ print(list(result))

#call the map function with the list of numbers to get the #list of results, as shown below.

```
numbers=(1, 2, 3, 4, 5)  
sqrList=map(square, numbers)
```

```
print(next(sqrList))  
print(next(sqrList))  
print(next(sqrList))
```

Types of Operator



Arithmetic Operators

43

Operator	Description	Example
** Exponent	Performs exponential (power) calculation on operators	$a^{**}b$ = 10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9//2 = 4$ $9.0//2.0 = 4.0$, $-11//3 = -4$, $-11.0//3 = -4.0$

Python Logical Operators

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is False.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is True.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is True.

Arithmetic Operators

44

```
x = 9
y = 2
# output: x + y = 11
print ('x + y =', x + y)
print ('x - y =', x - y)
print ('x * y =', x * y)
print ('x / y =', x / y)
print ( 'x // y =', x // y)
print ( 'x ** y =', x ** y)
```

Output:

```
x + y = 11
x - y = 7
x * y = 18
x / y = 4.5
x // y = 4
x ** y = 81
```

Python Logical Operators

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is False.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is True.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is True.

Python Membership Operators

45

- Python's membership operators test for membership in a sequence, such as strings, lists, or tuples. There are two membership operators as explained below –



Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

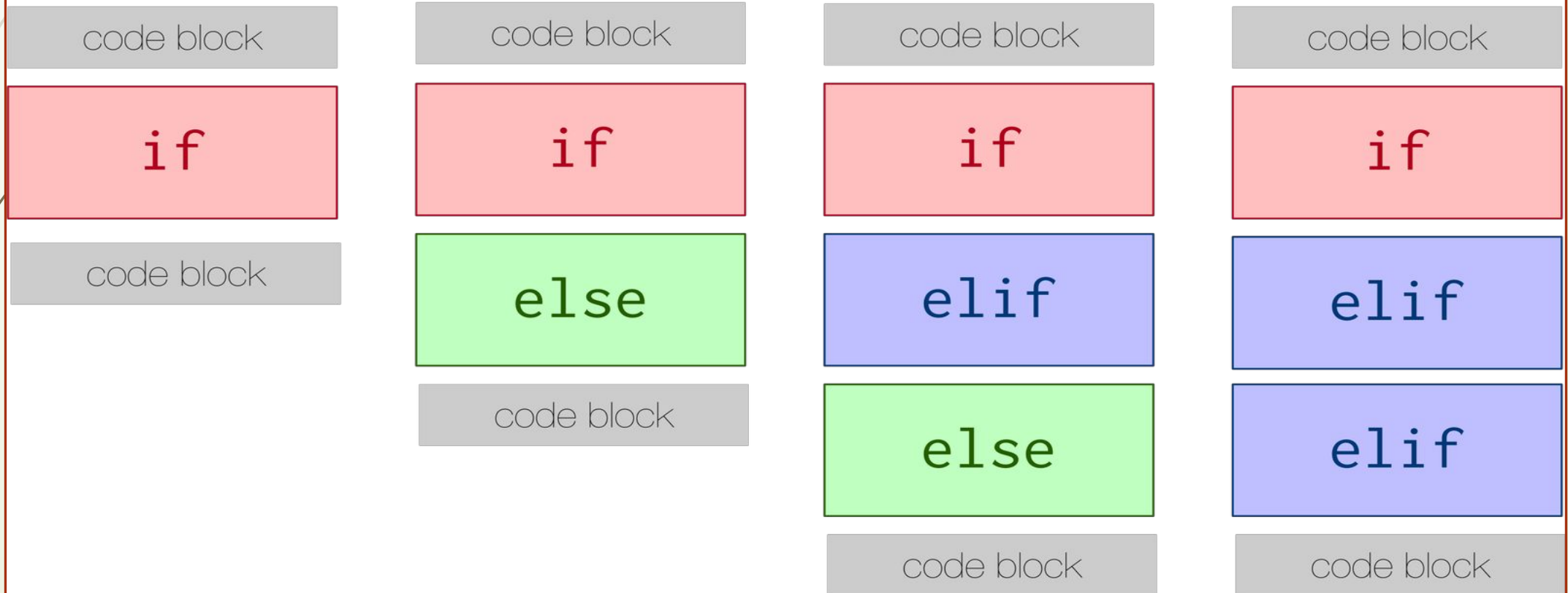
Python Identity Operators

- Identity operators compare the memory locations of two objects. There are two Identity operators as explained below –

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

Conditional Statements

valid if/elif/else order examples



Single Statement Suites

- ❑ If the suite of an if clause consists only of a single line, it may go on the same line as the header statement.

- ❑ **Example**

- ❑ Here is an example of a one-line if clause –

```
var = 100
```

```
if ( var == 100 ) :
```

```
    print ("Value of var is 100")
```

```
    print ("Good bye!")
```

- ❑ **Output**

- ❑ When the above code is executed, it produces the following result –

IF ELSE Statements

Syntax: >>> if(expression):

statements

else:

statements

- Python uses indentation to delimit the blocks of code (It doesn't use curly braces).
- If statements can be nested, one inside other. This means **inner if statement** in the statement part of the outer one. This is one way to see whether multiple conditions are satisfied.
- We can also use chain of if & else statements to determine which option in a series of possibilities is true.

elif Statements

- The "**elif**" (**Short for else if**) statement is a shortcut to use when chaining multiple if & else statements. A series of if elif statements can have a final else block, which is called if none of the if or elif conditions is true.
- Example :** >>> if(a>b): (statement) elif(b>a): (statement) elif(c>b): (statement) else(a>c) (statement)

In and not in

50

```
# Python program to illustrate
# Finding common member in list
# using 'in' operator
list1=[1,2,3,4,5]
list2=[6,7,8,9]
for item in list1:
    if item in list2:
        print("overlapping")
else:
    print("not overlapping")
```

```
# Python program to illustrate
# not 'in' operator
```

```
x = 24
y = 20
list = [10, 20, 30, 40, 50 ];
```

```
if ( x not in list ):
    print("x is NOT present in given list")
else:
    print("x is present in given list")
```

```
if ( y in list ):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

Is , is not

```
# Python program to illustrate the use  
# of 'is' identity operator  
x = 5  
if (type(x) is int):  
    print("true")  
else:  
    print("false")
```

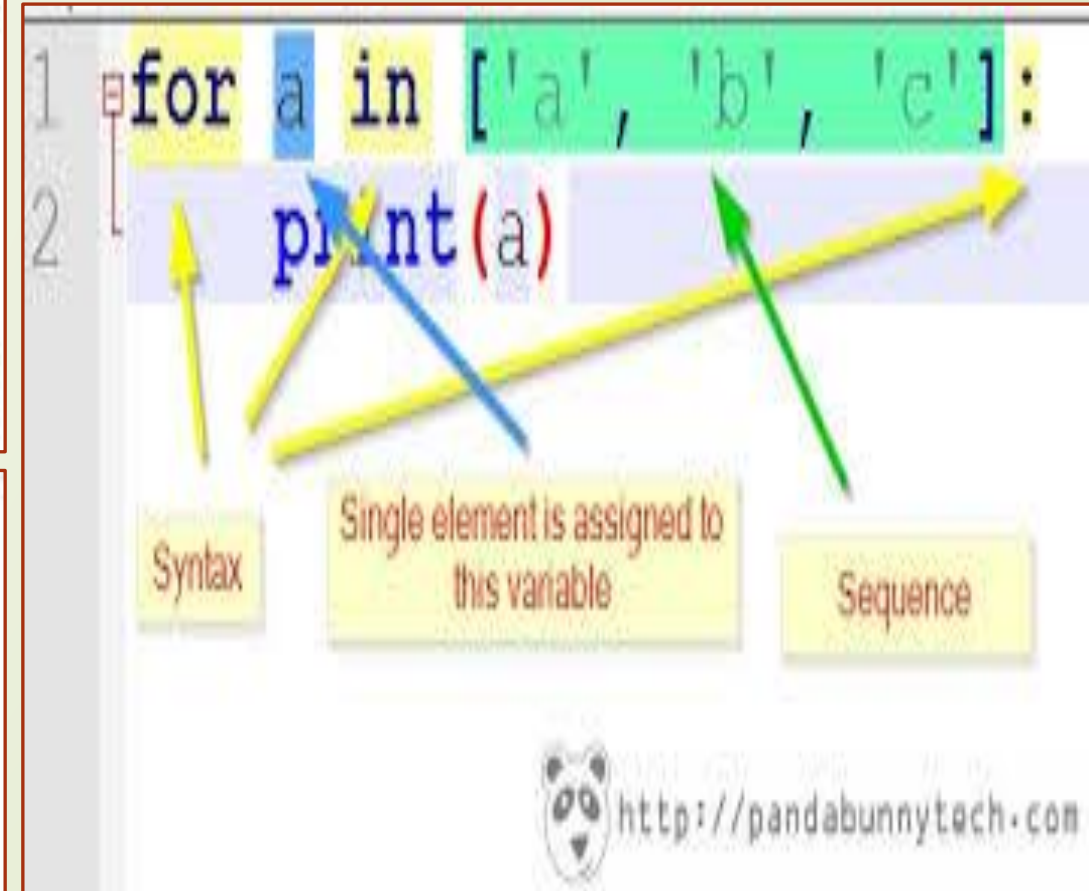
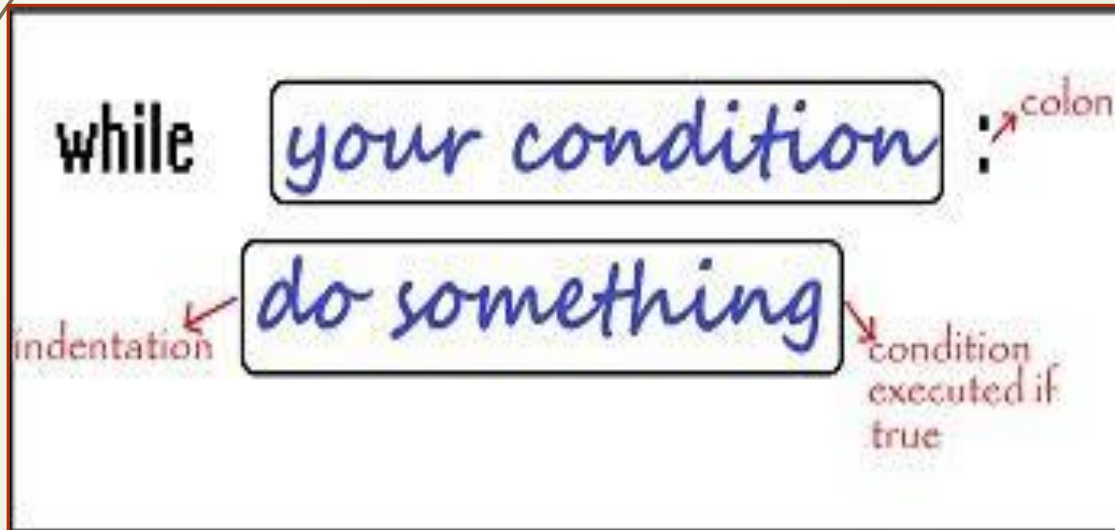
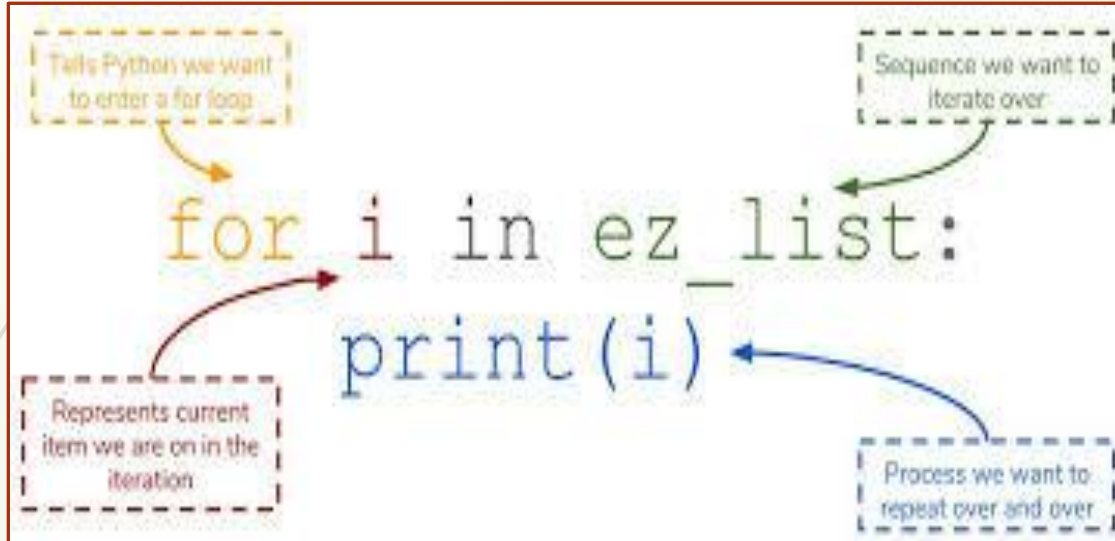
```
# Python program to illustrate the  
# use of 'is not' identity operator  
x = 5.2  
if (type(x) is not int):  
    print("true")  
else:  
    print("false")
```

Python programming language provides following types of loops to handle looping requirements.

52



Loop Syntax



Loop Control Statements



Sr.No.	Control Statement & Description
1	break statement Terminates the loop statement and transfers execution to the statement immediately following the loop.
2	continue statement Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
3	pass statement The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

```
for var in sequence:  
    # codes inside for loop  
    if condition:  
        continue  
    # codes inside for loop
```

```
# codes outside for loop
```

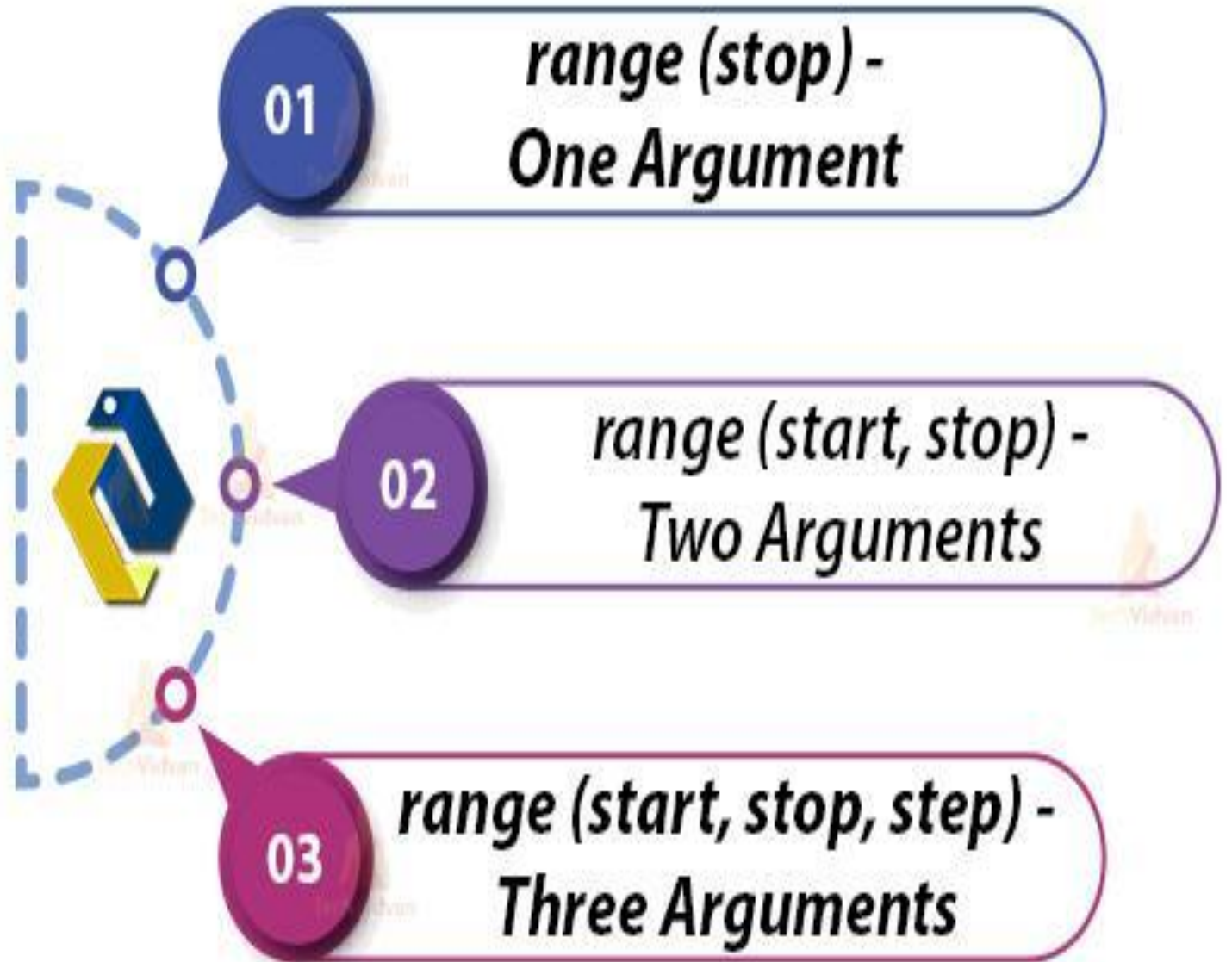
```
while test expression:  
    # codes inside while loop  
    if condition:  
        continue  
    # codes inside while loop  
  
# codes outside while loop
```

```
for var in sequence:  
    # codes inside for loop  
    if condition:  
        break  
    # codes inside for loop  
# codes outside for loop
```

```
while test expression:  
    # codes inside while loop  
    if condition:  
        break  
    # codes inside while loop  
# codes outside while loop
```

```
listA = [18, 21, 19, 29, 46]  
for val in listA:  
    pass  
  
def function(args):  
    pass  
  
class App:  
    pass
```

Python Range() Function



Example on control statements.

SYNTAX:

1. For var_name in range(limit):
statements
2. While (condition):
statements

Example:

For i in range(10):

print(i) #0....9

For i in range(n): #0....n-1

print(i)

i=1

While(i<=10):

print(i)

#program using while loop

n=1

cnt=1

while(cnt==1):

 print("token number",n)

 cnt=input("continue?(0/1)")

 cnt=int(cnt)

 n=n+1

print("Thank u, this is the end our day")

Breaking a Loop

You can use the break statement to break out of a loop early.

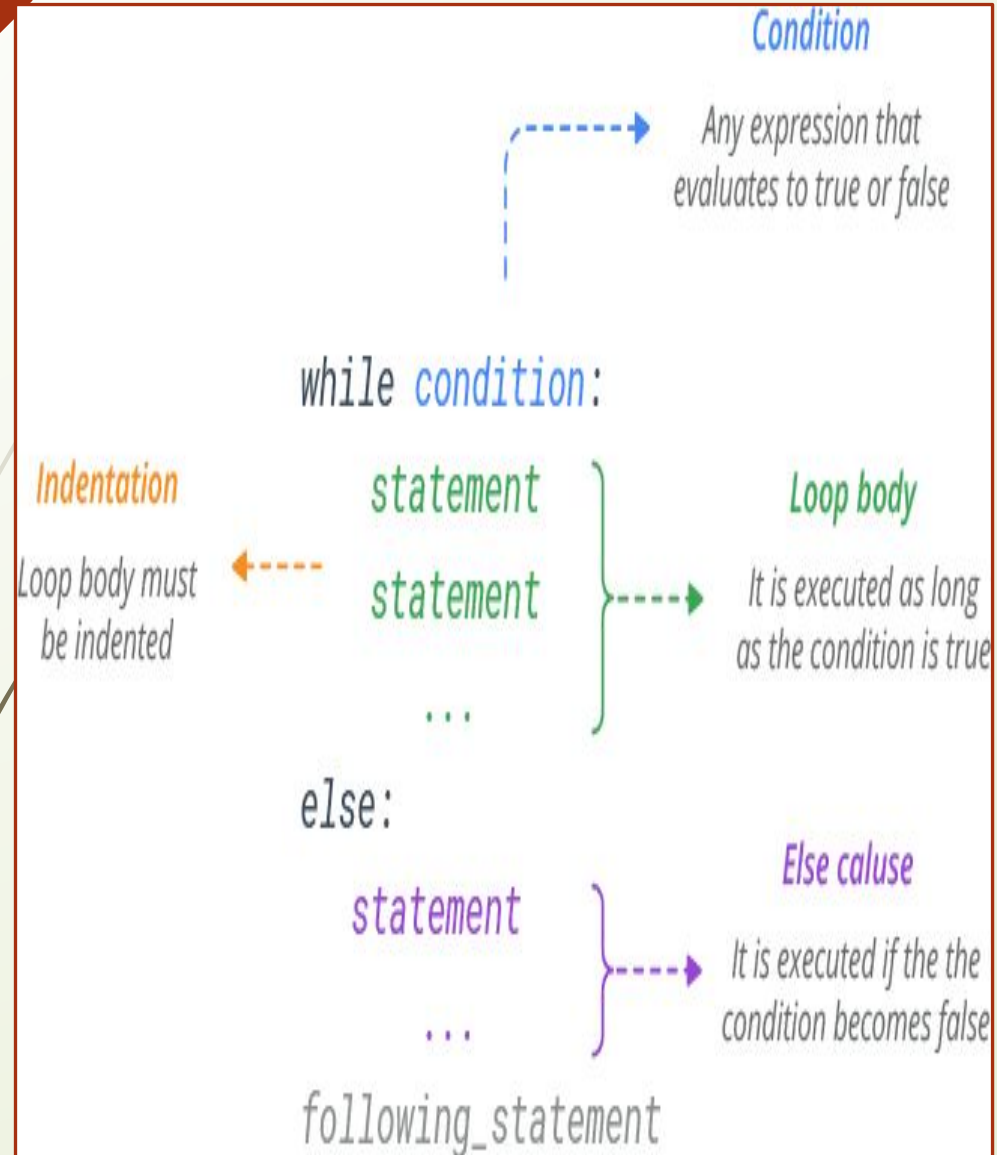
```
counter = 1
while (counter < 10):
    print (counter)
    counter = counter + 1
    if counter == 5:
        break
```

RESULT

1
2
3
4

A while Loop with else

60



- ❑ Python lets you add an else part to your loops. This is similar to using else with an if statement. It lets you specify what to do when/if the loop condition becomes false.
- ❑ `counter = 1`
- ❑ `while (counter < 10):`
- ❑ `print (counter)`
- ❑ `counter = counter +1`
- ❑ `else:`
- ❑ `print("The loop has successfully completed!")`

A for Loop with else

61

Python lets you add an else part to your loops. This is similar to using else with an if statement. It lets you specify what to do when the loop has finished.

```
planets = ["Earth", "Mars", "Neptune", "Venus", "Mercury", "Saturn", "Jupiter", "Uranus"]
for i in planets:
    print(i)
else:
    print("That's all folks!")
```

RESULT

Earth Mars Neptune Venus Mercury Saturn Jupiter Uranus That's all folks!

For loop with continue

```
□ # Program to show the use of continue statement inside
    loops
for val in "string":
    if val == "i":
        continue
    print(val)
print("The end")
```

OUTPUT:

s
t
r
i
n
g

```
for letter in 'Python':    # First Example
```

```
    if letter == 'h':
```

```
        continue
```

```
    print( 'Current Letter :', letter)
```

```
var = 10                # Second Example
```

```
while var > 0:
```

```
    var = var -1
```

```
    if var == 5:
```

```
        continue
```

```
    print ( 'Current variable value :', var)
```

```
print "Good bye!"
```

What is pass statement in Python?

- ❑ In Python programming, pass is a null statement. The difference between a comment and pass statement in Python is that, while the interpreter ignores a comment entirely, pass is not ignored.
- ❑ However, nothing happens when pass is executed. It results into no operation (NOP).

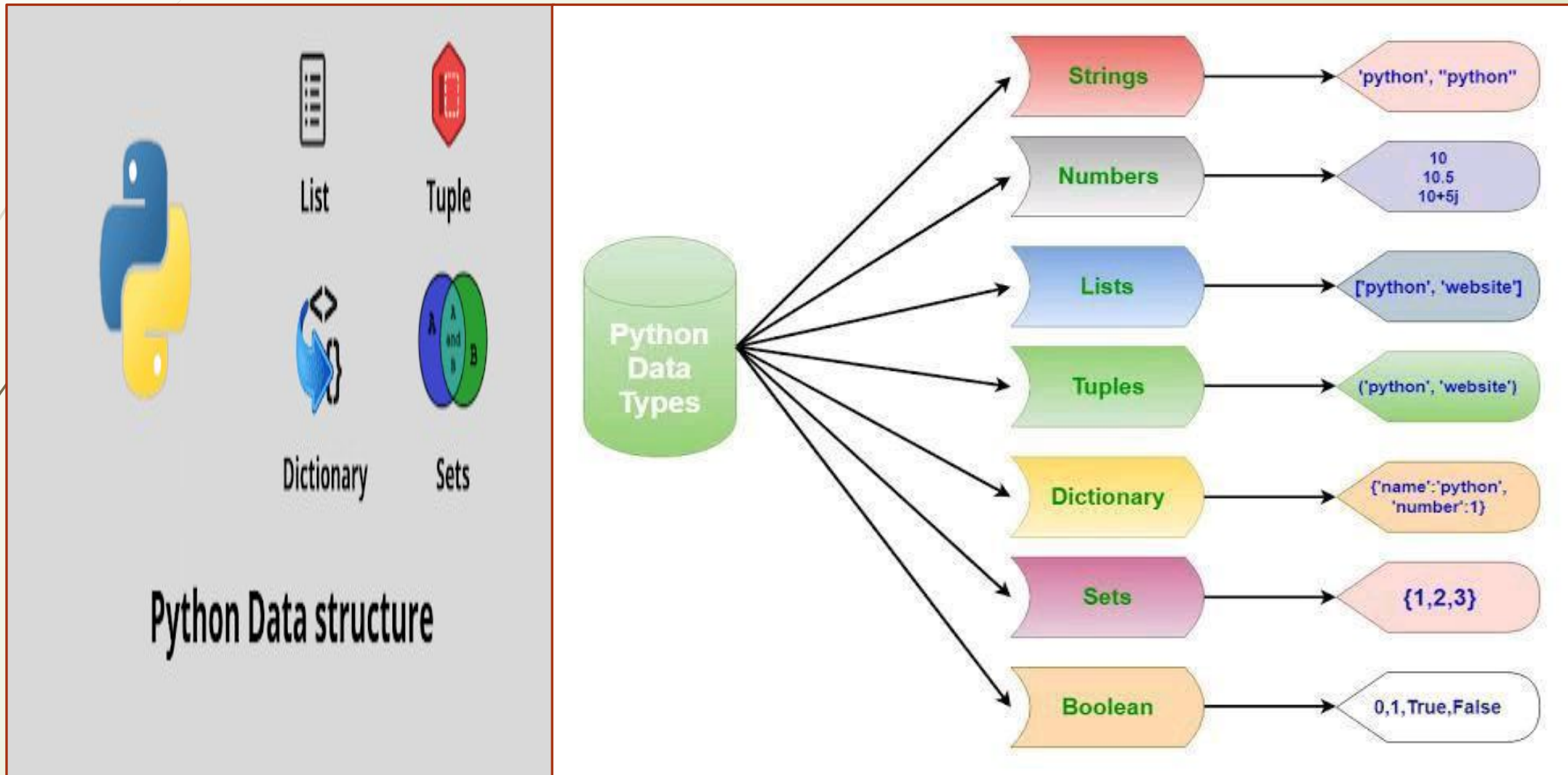
Syntax of pass

```
pass
# pass is just a placeholder for
# functionality to be added later.
sequence = {'p', 'a', 's', 's'}
for val in sequence:
    pass
```

We can do the same thing in an empty function or class as well.

```
def function(args):
    pass
class example:
    pass
```

Standard Data Structures



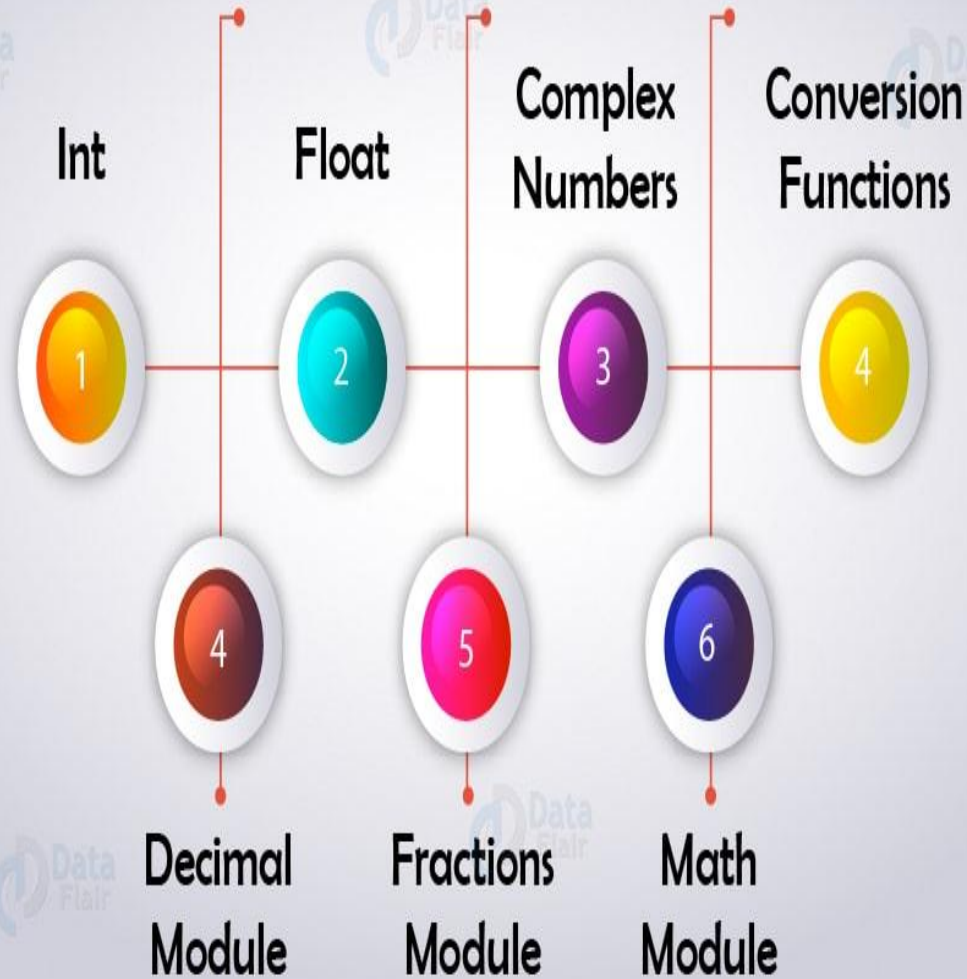
Standard Data Structures

66

- ❑ The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters.
- ❑ Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.
- ❑ **Python has standard data types –**
 - ❑ Numbers
 - ❑ String
 - ❑ List
 - ❑ Tuple
 - ❑ Dictionary
 - ❑ Set and boolean



Python Number Types



```
>>> 2+3j
```

```
(2+3j)
```

```
>>> type(2+3j)
```

```
<class 'complex'>
```

```
>>> 4.2 4.2
```

```
>>> (4.2)
```

```
<class 'float'>
```

```
>>> (10)
```

```
<class 'int'>
```

Mathematics Functions



Python Math Functions

69

Sr.No.	Function & Returns (description)
1	<code>abs(x)</code> The absolute value of x: the (positive) distance between x and zero.
2	<code>ceil(x)</code> The ceiling of x: the smallest integer not less than x
3	<code>exp(x)</code> The exponential of x: e^x
4	<code>fabs(x)</code> The absolute value of x.
5	<code>floor(x)</code> The floor of x: the largest integer not greater than x
6	<code>log(x)</code> The natural logarithm of x, for $x > 0$

8	<code>log10(x)</code> The base-10 logarithm of x for $x > 0$.
9	<code>max(x1, x2,...)</code> The largest of its arguments: the value closest to positive infinity
10	<code>min(x1, x2,...)</code> The smallest of its arguments: the value closest to negative infinity
11	<code>modf(x)</code> The fractional and integer parts of x in a two-item tuple. Both parts have the same sign as x . The integer part is returned as a float.
12	<code>pow(x, y)</code> The value of $x^{**}y$.
13	<code>round(x [,n])</code> x rounded to n digits from the decimal point. Python rounds away from zero as a tie-breaker: <code>round(0.5)</code> is 1.0 and <code>round(-0.5)</code> is -1.0.
14	<code>sqrt(x)</code> The square root of x for $x > 0$



Generate Random Numbers (int, float...) in Python

Random
Floats

`random()`
`uniform()`

Random
Objects

`choice()`
`shuffle()`

Random
Ints

`randint()`
`randrange()`



Random
`Choice()`

Random
`randrange()`

Random
`seed()`

Random
`shuffle()`

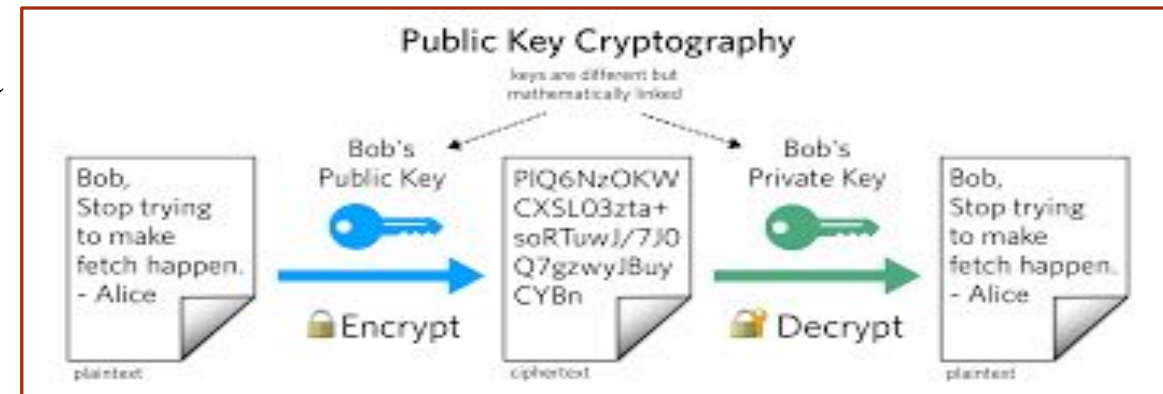
randint()

Python Random Numbers

Sr.No.	Function & Description
1	<code>choice(seq)</code> A random item from a list, tuple, or string.
2	<code>randrange ([start,] stop [,step])</code> A randomly selected element from <code>range(start, stop, step)</code>
3	<code>random()</code> A random float <code>r</code> , such that 0 is less than or equal to <code>r</code> and <code>r</code> is less than 1
4	<code>seed([x])</code> Sets the integer starting value used in generating random numbers. Call this function before calling any other random module function. Returns None.
5	<code>shuffle(lst)</code> Randomizes the items of a list in place. Returns None.
6	<code>uniform(x, y)</code> A random float <code>r</code> , such that <code>x</code> is less than or equal to <code>r</code> and <code>r</code> is less than <code>y</code>

Cryptographically secure random generator in Python

- ❑ Random Numbers and data generated by the random module are not cryptographically secure. So How to generate a random number that is cryptographically secure in Python?
- ❑ cryptographically secure pseudo-random number generator is a random number generator which has the **properties that make it suitable for use in cryptography application** where data security is essential.
- ❑ All cryptographically secure random generator function returns random bytes.
- ❑ Random bytes returned by this function depend on the random sources of the OS. Quality of randomness depends on random sources of the OS.
- ❑ We can use the following approaches to secure the random generator in Python cryptographically
- ❑ The **secrets** module to secure random data
- ❑ Use the `os.urandom()`
- ❑ use `random.SystemRandom` class





Numpy.random – PRNGs for Arrays

75

- ❑ PRNG is an acronym for pseudorandom number generator. As you know using the Python random module, we can generate scalar random numbers and data.
- ❑ Whenever you want to generate an array of random numbers you need to use numpy.random.
- ❑ numpy has the numpy.random package which has multiple functions to generate the random n-dimensional array for various distributions.
- ❑ Generate a random n-dimensional array of float numbers
- ❑ Use numpy.random.rand() to generate an n-dimensional array of random float numbers in the range of [0.0, 1.0)
- ❑ Use numpy.random.uniform to generate an n-dimensional array of random float numbers in the range of [low, high)

Numpy.random –EXAMPLE

76

- ❑ `import numpy`
- ❑ `#float`
- ❑ `random_float_array = numpy.random.rand(2, 2)`
- ❑ `print("2 X 2 random float array in [0.0, 1.0] \n", random_float_array, "\n")`
- ❑ `random_float_array = numpy.random.uniform(25.5, 99.5, size=(3, 2))`
- ❑ `print("3 X 2 random float array in range [25.5, 99.5] \n", random_float_array, "\n")`
- ❑ `#integers`
- ❑ `random_integer_array = numpy.random.random_integers(1, 10, 5)`
- ❑ `print("1-dimensional random integer array \n", random_integer_array, "\n")`
- ❑ `random_integer_array = numpy.random.random_integers(1, 10, size=(3, 2))`
- ❑ `print("2-dimensional random integer array \n", random_integer_array)`

Generate random Universally unique IDs

77

- ❑ Python UUID Module provides immutable UUID objects. UUID is a Universally Unique Identifier.
- ❑ It has the functions to generate all versions of UUID. Using `uuid.uuid4()` function, you can generate a 128 bit long random unique ID and it's cryptographically safe.
- ❑ These unique IDs are used to identify the documents, Users, resources or any information in computer systems. Learn more on the UUID module.
- ❑ Example:
 - ❑ `import uuid`
 - ❑ `# get a random UUID`
 - ❑ `safeId = uuid.uuid4()`
 - ❑ `print("safe unique id is ", safeId)`

Python Strings

- Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows either pair of single or double quotes. Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 to the end.
- The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator.

- **For example –**

```
str = 'Hello World!'
```

```
print (str)      # Prints complete string
```

```
print (str[0])   # Prints first character of the string
```

```
print (str[2:5]) # Prints characters starting from 3rd to 5th
```

```
print (str[2:])  # Prints string starting from 3rd character
```

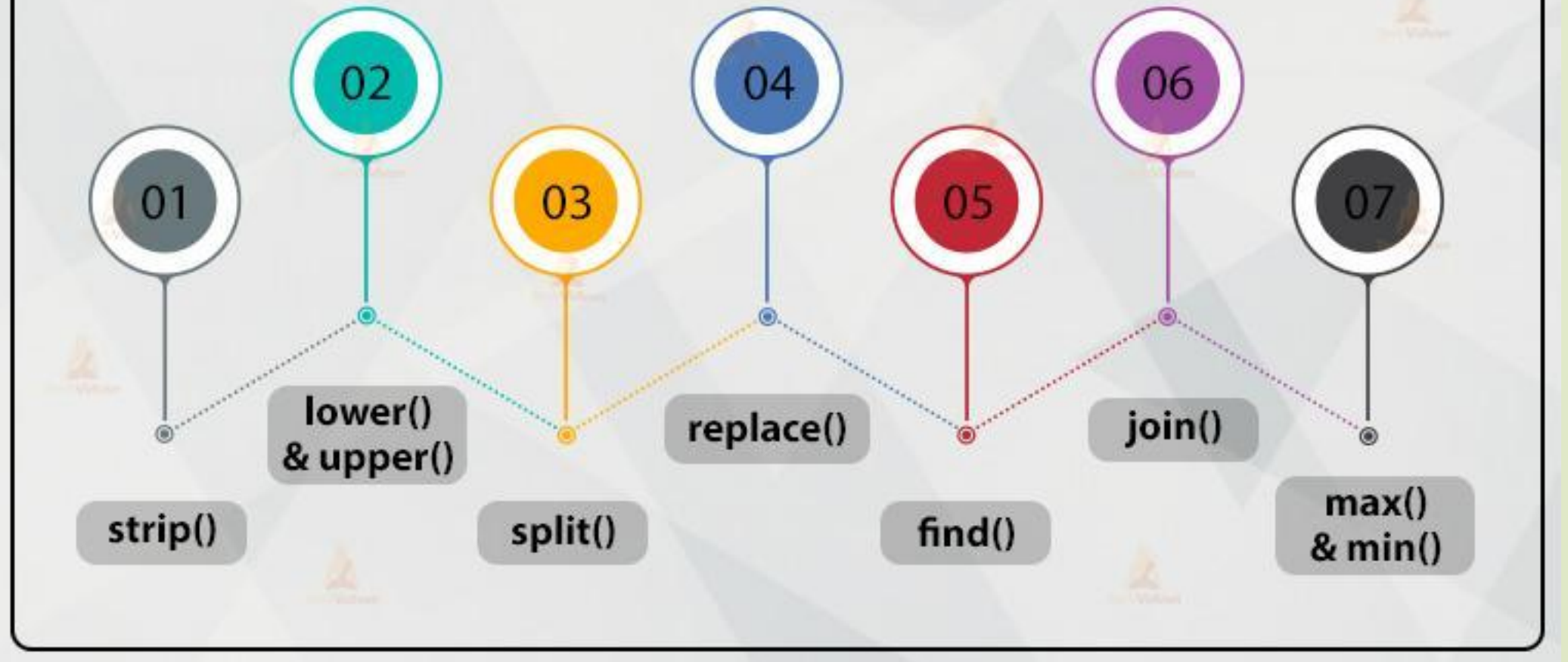
```
print (str * 2)  # Prints string two times
```

```
print (str + "TEST") # Prints concatenated string
```

H	e	l	l	o
0	1	2	3	4
-5	-4	-3	-2	-1



PYTHON STRING METHODS & FUNCTIONS



Python Lists

- ❑ Lists are the most versatile of Python's compound data types. A list contains items separated by commas and enclosed within square brackets ([]). To some extent, lists are similar to arrays in C. One of the differences between them is that all the items belonging to a list can be of different data type.
- ❑ The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1. The plus (+) sign is the list concatenation operator, and the asterisk (*) is the repetition operator.
- ❑ For example –

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tinylist = [123, 'john']
print (list)           # Prints complete list
print (list[0])        # Prints first element of the list
print (list[1:3])      # Prints elements starting from 2nd till
3rd
print (list[2:])        # Prints elements starting from 3rd
element
print (tinylist * 2)    # Prints list two times
print (list + tinylist) # Prints concatenated lists
```

PYTHON LIST METHODS

append()

extend()

insert()

remove()

pop()

clear()

index()

count()

sort()

reverse()

copy()

Make a copy of a list with the copy() method:

```
❑ thislist = ["apple", "banana", "cherry"]  
    mylist = thislist.copy()  
    print(mylist)
```

❑ #Sort list elements:

```
    num=int(input())  
    arr=list()  
    for i in range(num):  
        ele=int(input())  
        arr.append(ele)  
    arr.sort()  
    print(*arr,sep=" ")
```

Built-In functions

Method	Description	Examples
<code>append(x)</code>	Adds an item (x) to the end of the list. This is equivalent to <code>a[len(a):] = [x]</code> .	<pre>a = ["bee", "moth"] print(a) a.append("ant") print(a)</pre> RESULT <pre>['bee', 'moth'] ['bee', 'moth', 'ant']</pre>
<code>extend(iterable)</code>	Extends the list by appending all the items from the iterable. This allows you to join two lists together. This method is equivalent to <code>a[len(a):] = iterable</code> .	<pre>a = ["bee", "moth"] print(a) a.extend(["ant", "fly"]) print(a)</pre> RESULT <pre>['bee', 'moth'] ['bee', 'moth', 'ant', 'fly']</pre>

Built-In functions continued...

`insert(i, x)`

Inserts an item at a given position. The first argument is the index of the element before which to insert. For example, `a.insert(0, x)` inserts at the front of the list.

```
a = ["bee", "moth"]
print(a)
a.insert(0, "ant")
print(a)
a.insert(2, "fly")
print(a)
```

RESULT

```
['bee', 'moth']
['ant', 'bee', 'moth']
['ant', 'bee', 'fly', 'moth']
```

`remove(x)`

Removes the first item from the list that has a value of `x`. Returns an error if there is no such item.

```
a = ["bee", "moth", "ant"]
print(a)
a.remove("moth")
print(a)
```

RESULT

```
['bee', 'moth', 'ant']
['bee', 'ant']
```

Built-In functions continued

85

`pop([i])`

Removes the item at the given position in the list, and returns it. If no index is specified, `pop()` removes and returns the last item in the list.

```
# Example 1: No index specified
a = ["bee", "moth", "ant"]
print(a)
a.pop()
print(a)
```

```
# Example 2: Index specified
a = ["bee", "moth", "ant"]
print(a)
a.pop(1)
print(a)
```

RESULT

```
['bee', 'moth', 'ant']
['bee', 'moth']
['bee', 'moth', 'ant']
['bee', 'ant']
```

Built-In functions continued

`clear()`

Removes all items from the list.
Equivalent to `del a[:]`.

```
a = ["bee", "moth",  
     "ant"]  
print(a)  
a.clear()  
print(a)
```

RESULT

```
['bee', 'moth', 'ant']  
[]
```

`index(x[,
start[, end]])`

Returns the position of the first
list item that has a value of `x`.
Raises a `ValueError` if there is
no such item.

The optional arguments `start`
and `end` are interpreted as in
the slice notation and are used

```
a = ["bee", "ant",  
     "moth", "ant"]  
print(a.index("ant"))  
print(a.index("ant",  
              2))
```

RESULT

```
1  
2
```

Built-In functions continued

no such item.

The optional arguments `start` and `end` are interpreted as in the slice notation and are used to limit the search to a particular subsequence of the list. The returned index is computed relative to the beginning of the full sequence rather than the start argument.

```
print(a.index("ant",  
2))
```

RESULT

1
3

`count(x)`

Returns the number of times `x` appears in the list.

```
a = ["bee", "ant",  
     "moth", "ant"]  
print(a.count("bee"))  
print(a.count("ant"))  
print(a.count(""))
```

RESULT

1
2

Built-In functions continued

88

```
sort(key=None,  
reverse=False)
```

Sorts the items of the list in place. The arguments can be used to customize the operation.

key

Specifies a function of one argument that is used to extract a comparison key from each list element. The default value is `None` (compares the elements directly).

reverse

Boolean value. If set to `True`, then the list elements are sorted as if each comparison were reversed.

```
a = [3,6,5,2,4,1]
a.sort()
print(a)

a = [3,6,5,2,4,1]
a.sort(reverse=True)
print(a)

a = ["bee", "wasp", "moth", "ant"]
a.sort()
print(a)

a = ["bee", "wasp", "butterfly"]
a.sort(key=len)
print(a)

a = ["bee", "wasp", "butterfly"]
a.sort(key=len, reverse=True)
print(a)
```

RESULT

```
[1, 2, 3, 4, 5, 6]
[6, 5, 4, 3, 2, 1]
['ant', 'bee', 'moth', 'wasp']
['bee', 'wasp', 'butterfly']
['butterfly', 'wasp', 'bee']
```

RESULT

`copy()`

Returns a shallow copy of the list. Equivalent to `a[:]`.



Use the `copy()` method when you need to update the copy without affecting the original list. If you don't use this method (eg, if you do something like `list2 = list1`), then any updates you do to `list2` will also affect `list1`.

The example at the side demonstrates this.

```
# WITHOUT copy()
a = ["bee", "wasp", "moth"]
b = a
b.append("ant")
print(a)
print(b)

# WITH copy()
a = ["bee", "wasp", "moth"]
b = a.copy()
b.append("ant")
print(a)
print(b)
```

RESULT

```
['bee', 'wasp', 'moth', 'ant']
['bee', 'wasp', 'moth', 'ant']
['bee', 'wasp', 'moth']
['bee', 'wasp', 'moth', 'ant']
```

`len(s)`

Returns the number of items in the list.



The `len()` function can be used on any sequence (such as a string, bytes, tuple, list, or range) or collection (such as a dictionary, set, or frozen set).

```
a = ["bee", "moth", "ant"]  
print(len(a))
```

RESULT

3

Built-In functions continued

91

`list([iterable])`

The `list()` constructor returns a mutable sequence list of elements. The *iterable* argument is optional. You can provide any sequence or collection (such as a string, list, tuple, set, dictionary, etc). If no argument is supplied, an empty list is returned.



Strictly speaking, `list([iterable])` is actually a mutable sequence type.

```
print(list())
print(list([]))
print(list(["bee", "moth",
"ant"]))
print(list([[ "bee",
"moth"], [ "ant"]]))
```

```
a = "bee"
print(list(a))
```

```
a = ("I", "am", "a",
"tuple")
print(list(a))
```

```
a = {"I", "am", "a",
"set"}
print(list(a))
```

RESULT

```
[]
[]
['bee', 'moth', 'ant']
[['bee', 'moth'],
['ant']]
['b', 'e', 'e']
['I', 'am', 'a', 'tuple']
```

Built-In functions continued

92

```
max(iterable, *  
[, key, default])
```

or

```
max(arg1, arg2,  
*args[, key])
```

Returns the largest item in an iterable (eg, list) or the largest of two or more arguments.

The *key* argument specifies a one-argument ordering function like that used for `sort()`.

The *default* argument specifies an object to return if the provided iterable is empty. If the iterable is empty and *default* is not provided, a `ValueError` is raised.

If more than one item shares the maximum value, only the first one encountered is returned.

```
a = ["bee", "moth", "ant"]  
print(max(a))
```

```
a = ["bee", "moth",  
"wasp"]  
print(max(a))
```

```
a = [1, 2, 3, 4, 5]  
b = [1, 2, 3, 4]  
print(max(a, b))
```

RESULT

```
moth  
wasp  
[1, 2, 3, 4, 5]
```

Built-In functions continued

```
min(iterable, *  
[, key, default])
```

or

```
min(arg1, arg2,  
*args[, key])
```

Returns the smallest item in an iterable (eg, list) or the smallest of two or more arguments.

The *key* argument specifies a one-argument ordering function like that used for `sort()`.

The *default* argument specifies an object to return if the provided iterable is empty. If the iterable is empty and *default* is not provided, a `ValueError` is raised.

If more than one item shares the minimum value, only the first one encountered is returned.

```
a = ["bee", "moth",  
"wasp"]  
print(min(a))
```

```
a = ["bee", "moth", "ant"]  
print(min(a))
```

```
a = [1, 2, 3, 4, 5]  
b = [1, 2, 3, 4]  
print(min(a, b))
```

RESULT

```
bee  
ant  
[1, 2, 3, 4]
```

Built-In functions continued

94

`range(stop)`

or

`range(start,
stop[, step])`

Represents an immutable sequence of numbers and is commonly used for looping a specific number of times in `for` loops.

It can be used along with `list()` to return a list of items between a given range.



Strictly speaking, `range()` is actually a mutable sequence type.

```
print(list(range(10)))  
print(list(range(1,11)))  
print(list(range(51,56)))  
print(list(range(1,11,2)))
```

RESULT

```
[0, 1, 2, 3, 4, 5, 6, 7,  
8, 9]  
[1, 2, 3, 4, 5, 6, 7, 8,  
9, 10]  
[51, 52, 53, 54, 55]  
[1, 3, 5, 7, 9]
```

Python Tuples

95

- A tuple is another sequence data type that is similar to the list. A tuple consists of a number of values separated by commas. Unlike lists, however, tuples are enclosed within parenthesis.
- The main difference between lists and tuples are – Lists are enclosed in brackets ([]) and their elements and size can be changed, while tuples are enclosed in parentheses (()) and cannot be updated.
- Tuples can be thought of as **read-only** lists.
- For example –

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
tinytuple = (123, 'john')
print (tuple)          # Prints complete tuple
print (tuple[0])       # Prints first element of the tuple
print (tuple[1:3])     # Prints elements starting from
2nd till 3rd
print (tuple[2:])      # Prints elements starting from
3rd element
print (tinytuple * 2)  # Prints tuple two times
print (tuple + tinytuple) # Prints concatenated tuple
```



PYTHON TUPLES VS LISTS

TUPLES

The items are surrounded in paranthesis ().

Tuples are immutable in nature.

There are 33 available methods on tuples.

In dictionary, we can create keys using tuples.

Syntax

Mutability

Methods

Usability

LISTS

The items are surrounded in square brackets [].

Lists are mutable in nature.

There are 46 available methods on lists.

In dictionary, we can't use lists as keys.



Python Tuple Functions

len()

max()

min()

sum()

any()

all()

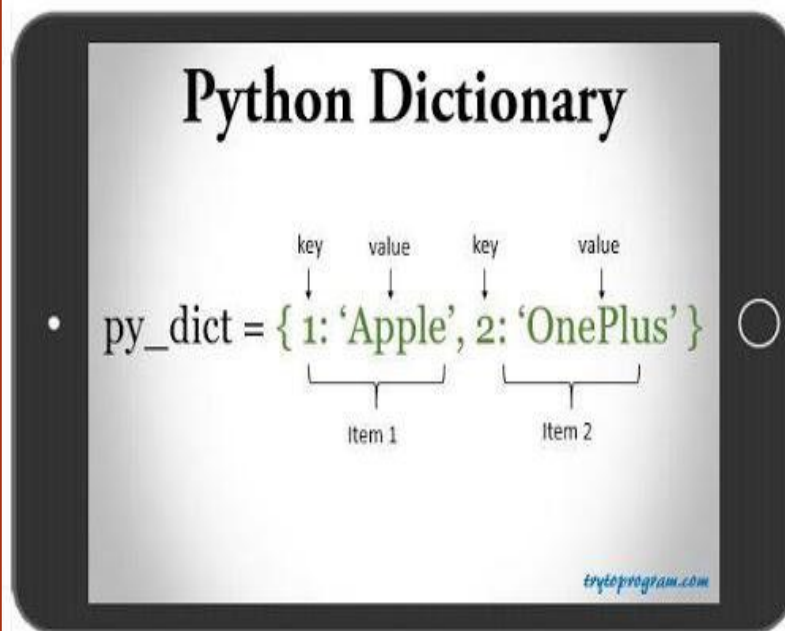
sorted()

tuple()

Python Dictionary

98

- ❑ Python's dictionaries are kind of hash-table type. They work like associative arrays or hashes found in Perl and consist of key-value pairs.
- ❑ A dictionary key can be almost any Python type, but are usually numbers or strings. Values, on the other hand, can be any arbitrary Python object.
- ❑ Dictionaries are enclosed by curly braces ({ }) and values can be assigned and accessed using square braces ([]).



#program

```
dict = {} #empty dictionary
```

```
tinydict = {'name': 'john', 'code': 6734, 'dept': 'sales'}
```

```
dict['one'] = "This is one"
```

```
dict[2] = "This is two"
```

```
#dict={'one': 'this is one', 2: 'this is two'}
```

```
print (dict['one']) # Prints value for 'one' key
```

```
print (dict[2]) # Prints value for 2 key
```

```
print (tinydict) # Prints complete dictionary
```

```
print (tinydict.keys()) # Prints all the keys
```

```
print (tinydict.values()) # Prints all the values
```

Python Dictionary

99

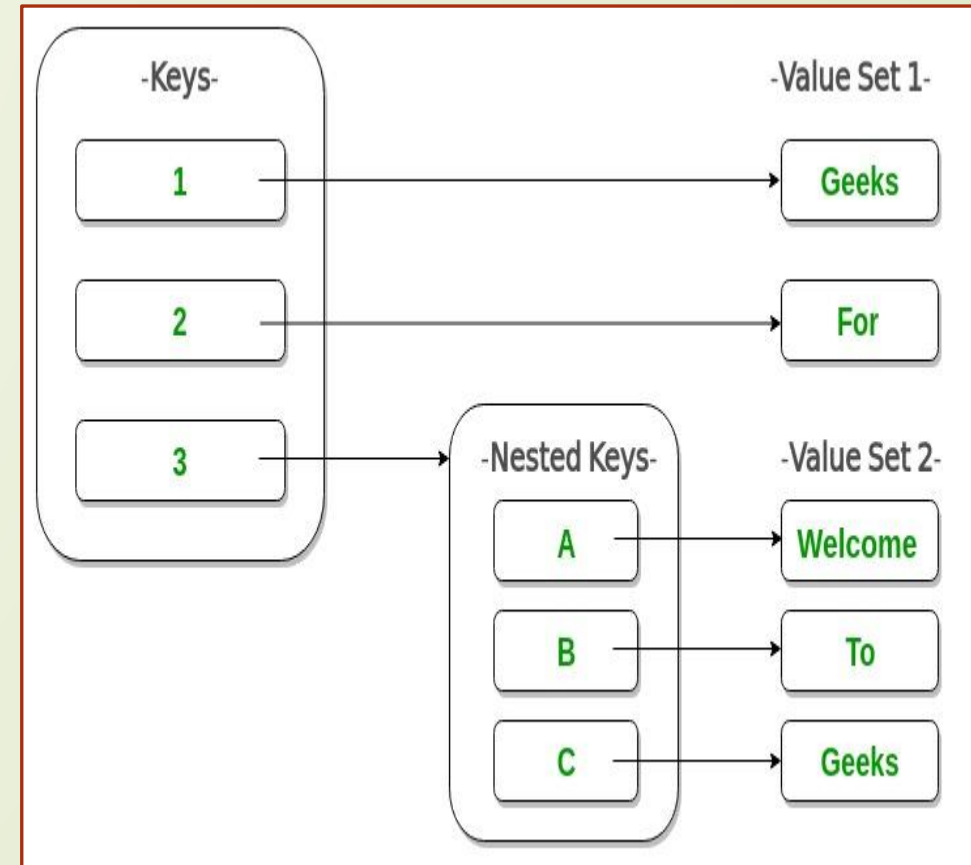
Properties of Dictionary Keys

- There are two important points while using dictionary keys

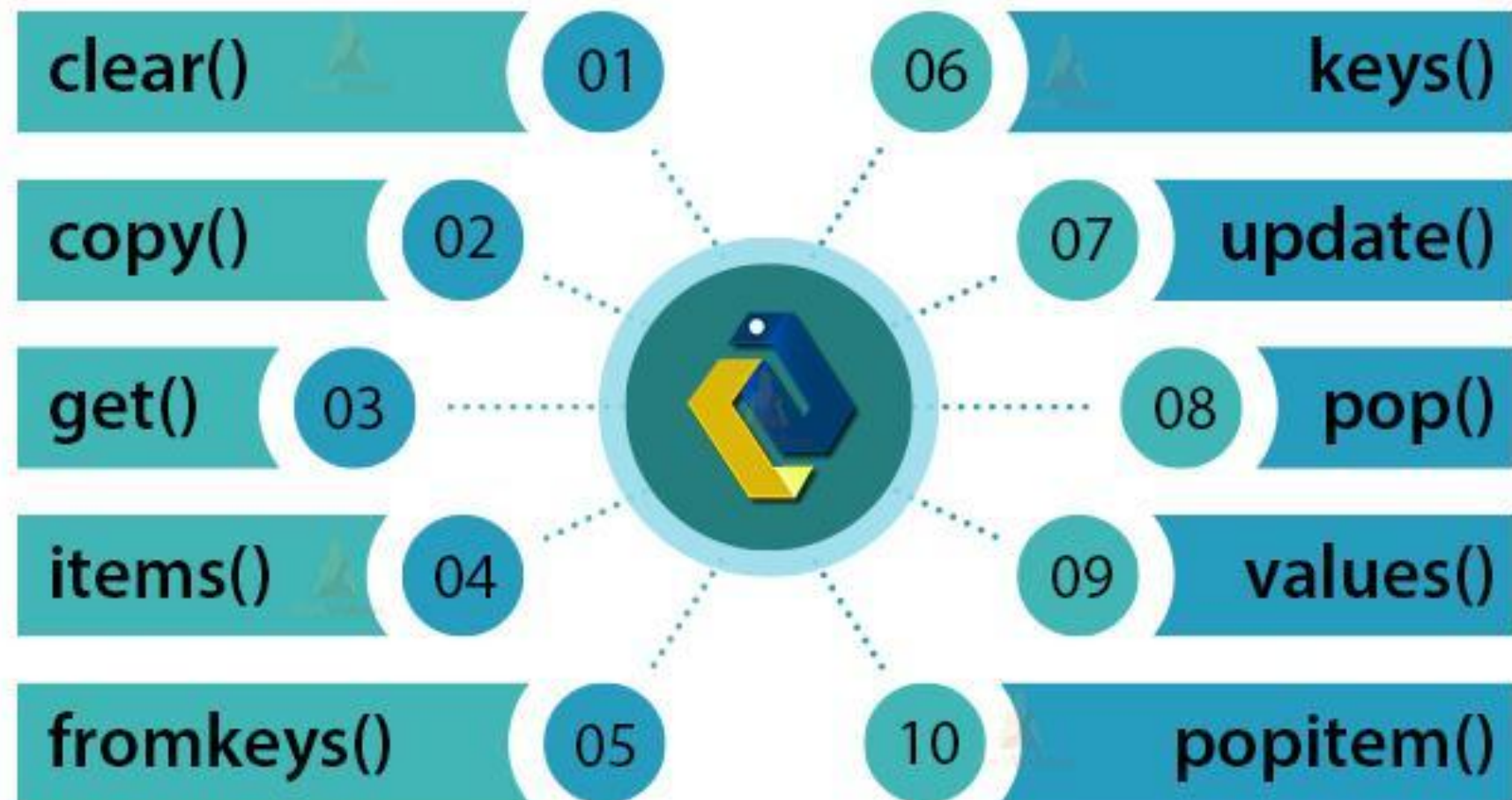
More than one entry per key is not allowed (no duplicate key is ~~not~~ allowed).
`d={1:2,2:4,2:6,3:8,3:4,2:10}`

- The values in the dictionary can be of any type while the keys must be immutable tuples.
- Dictionary keys are case sensitive- Same key name but with the different case are treated as different keys in Python dictionaries.

`dict = {42: 'aaa', 2.78: 'bbb', True: 'ccc'}`



Python Dictionary Methods



Python Set:

`set('Python')`
↓
{`'t'`, `'o'`, `'y'`, `'n'`, `'h'`, `'P'`}

© w3resource.com

- ❑ A set is an unordered collection of items. **Every set element is unique (no duplicates)** and must be immutable (cannot be changed).
- ❑ Sets can also be used to perform mathematical set operations like **union, intersection, difference, etc.**
- ❑ A set is created by placing all the items (elements) inside curly **braces {}**, separated by **comma**, or by using the built-in `set()` function.
- ❑ It can have any number of items and they may be of **different types (integer, float, tuple, string etc.)**. But a **set cannot have mutable elements** like lists or dictionaries as its elements.
- ❑ `set()` method is used to convert any of the iterable to sequence of iterable elements with distinct elements, commonly called Set.
- ❑ Parameters : Any iterable sequence like list, tuple or dictionary. Returns : **An empty set if no element is passed**. Non-repeating element iterable modified as passed as argument.

Python Set

102

- ❑ # Distinguish set and dictionary while creating empty set
- ❑ # initialize a with {}
- ❑ a = {}
- ❑ # check data type of a
- ❑ print(type(a)) #dict
- ❑ # initialize a with set()
- ❑ a = set() # set()
- ❑ # check data type of a
- ❑ print(type(a))

set cannot have duplicates

#

```
my_set = {1, 2, 3, 4, 3, 2}  
print(my_set)
```

we can make set from a list

```
my_set = set([1, 2, 3, 2])  
print(my_set)
```

Python Set

103

- ❑ # Different types of sets in Python
- ❑ # set of integers
- ❑ my_set = {1, 2, 3}
- ❑ print(my_set)
- ❑ # set of mixed datatypes
- ❑ my_set = {1.0, "Hello", (1, 2, 3)}
- ❑ print(my_set)

```
# initialize my_set  
my_set = {1, 3}  
print(my_set)
```

```
my_set.add(2)  
print(my_set)
```

```
# add multiple elements  
# Output: {1, 2, 3, 4}  
my_set.update([2, 3, 4])  
print(my_set)
```

```
# add list and set  
# Output: {1, 2, 3, 4, 5, 6, 8}  
my_set.update([4, 5], {1, 6, 8})  
print(my_set)
```

❑ Removing elements from a set

- ❑ A particular item can be removed from a set using the methods `discard()` and `remove()`.
- ❑ The only difference between the two is that the `discard()` function leaves a set unchanged if the element is not present in the set. On the other hand, the `remove()` function will raise an error in such a condition (if element is not present in the set).
- ❑ The following example will illustrate this.

Difference between `discard()` and `remove()`

initialize my_set

```
my_set = {1, 3, 4, 5, 6}
```

```
print(my_set)
```

discard an element

Output: {1, 3, 5, 6}

```
my_set.discard(4)
```

```
print(my_set)
```

remove an element

Output: {1, 3, 5}

```
my_set.remove(6)
```

```
print(my_set)
```

discard an element

not present in my_set

Output: {1, 3, 5}

```
my_set.discard(2)
```

```
print(my_set)
```

```
# remove an element  
# not present in my_set  
# you will get an error.  
# Output: KeyError
```

```
my_set.remove(2)
```

```

❑ # initialize my_set
❑ # Output: set of unique elements
❑ my_set = set("HelloWorld")
❑ print(my_set)
❑ # pop an element
❑ # Output: random element
❑ print(my_set.pop())
❑ # pop another element
❑ my_set.pop()
❑ print(my_set)
❑ # clear my_set
❑ # Output: set()
❑ my_set.clear()
❑ print(my_set)
❑ print(my_set)

```

```

# Set union method
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use | operator
# Output: {1, 2, 3, 4, 5, 6, 7, 8}
print(A | B)

```

```

# use union function
>>> A.union(B)
{1, 2, 3, 4, 5, 6, 7, 8}

# use union function on B
>>> B.union(A)
{1, 2, 3, 4, 5, 6, 7, 8}

```

```

# Intersection of sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use & operator
# Output: {4, 5}
print(A & B)

```

```

# use intersection function on A
>>> A.intersection(B)
{4, 5}

# use intersection function on B
>>> B.intersection(A)
{4, 5}

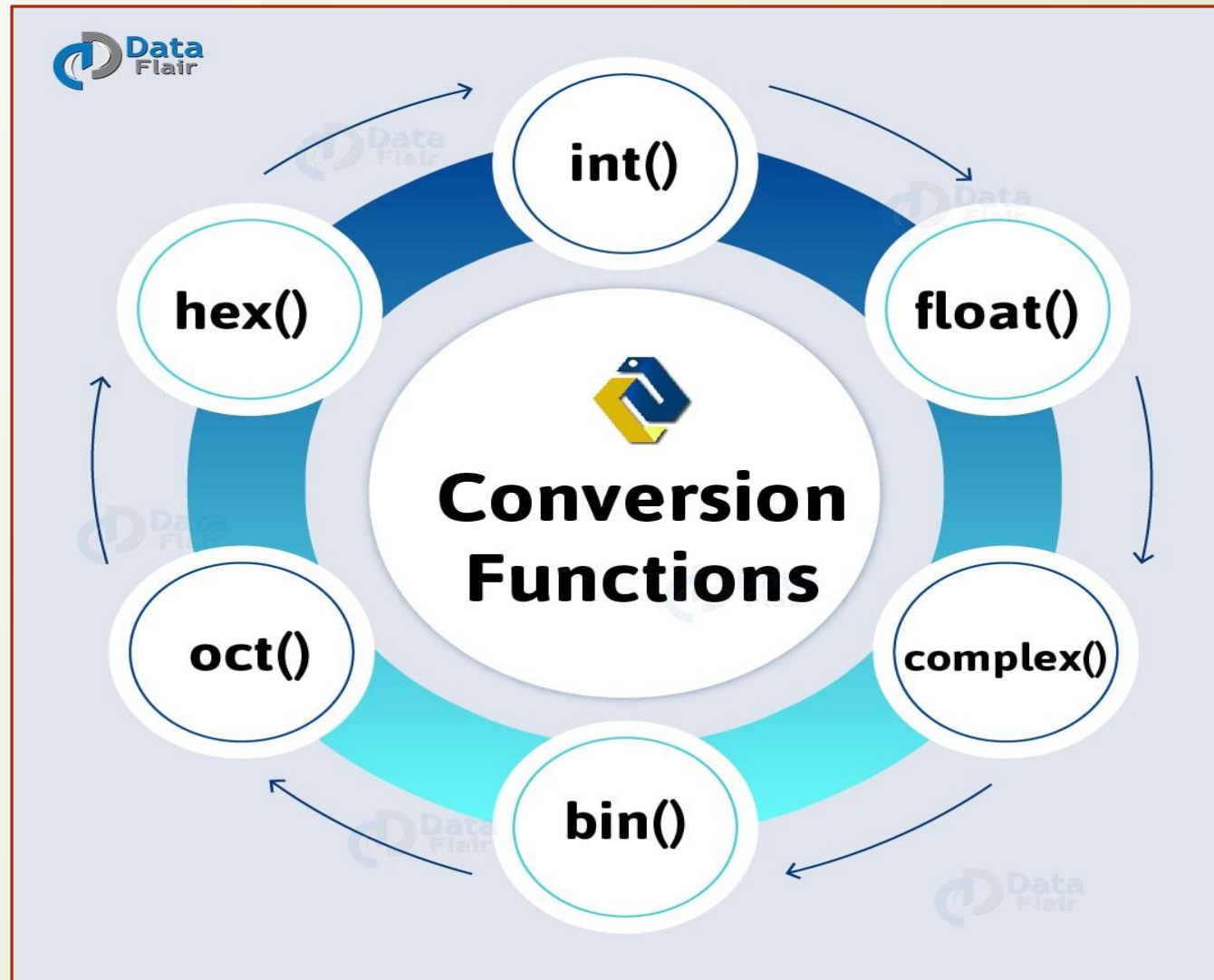
```

```

# Difference of two sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use - operator on A
# Output: {1, 2, 3}
print(A - B)

```



TRY

107

Anagram



- Write a Python function that takes two lists and returns True if they have at least one common member.

Panagram



Write a Python program to get the frequency of the elements in a list.

Program on Anagram.

```

str1 = input("enter the first string:")
str2 = input("enter second string:")
count1 = 0
i=0
while(i<len(str1)):
    count1 = count1 +ord(str1[i])
    i=i+1
count2=0
i=0
while(i<len(str2)):
    count2 = count2 +ord(str2[i])
    i=i+1
if(count1==count2):
    print("anagram")
else:
    print("not anagram")

```

Output : listen
Listen
anagram

LISTEN
SILENT

Program on panagram

```

def checkpanagram(s):
    list = []
    for i in range(26):
        list.append(False)
    for c in s.lower():
        if not c == " ":
            list[ord(c) - ord('a')] = True
    for ch in list:
        if ch== False:
            return False
    return True
sentence= input("Enter any sentence:")
if (checkpanagram(sentence)):
    print('Yes', end=' ')
else:
    print('No', end= ' ')

```

Fred
specialized
in the job of
making very
quaint wax
toys.

DEFINING USER DEFINED FUNCTION

109

Syntax

```
def functionname( [parameters ]):  
    "function_docstring" function_suite  
    return [expression]
```

By default, parameters have a positional behavior and you need to inform them in the same order that they were defined.

Example 1:

```
def my_function():  
    print("Hello from a function")  
my_function() # call function
```

Example 2:

The following function takes a string as input parameter and prints it on standard screen.

```
def printme( str ):  
    #This prints a passed string into  
    this function  
    print(str )  
printme("python") # calling
```

□ Parameters

- Information can be passed to functions as parameter.
- Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.
- The following example has a function with one parameter (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

□ Example

```
□ def my_function(fname):  
    print(fname + ", WELCOME TO PYTHON PRGRAMMING")
```

```
my_function("sonu")  
my_function("sam")  
my_function("ram")
```

Examples on Functions

```
def my_func():  
    x = 10  
    print("Value inside function:",x)  
x = 20  
my_func()  
print("Value outside function:",x)
```

```
def my_function(country = "Norway"):  
    print("I am from " + country)
```

```
my_function("Sweden")  
my_function("India")  
my_function()  
my_function("Brazil")
```

```
def my_function(food):  
    print(food, end="")
```

```
fruits = ["apple", "banana", "cherry"]  
  
my_function(fruits)
```

Function by generating Calendar

```
# Program to display calendar of the given month and year
import calendar
def cal(yy,mm):
    # display the calendar
    print(calendar.month(yy, mm))
# To take month and year input from the user
yy = int(input("Enter year: "))
mm = int(input("Enter month: "))
cal(yy,mm)
```

Function with list as a parameter

```
def swap(x, y):  
    temp = x;  
    x = y;  
    y = temp;
```

Driver code

```
x = 2  
y = 3  
swap(x, y)  
print(x)  
print(y)
```

#FUNCTION WITH RETURN VALUE

```
def square(x):  
    return(x*x)
```

print(square(4)) **#calling function**

square(9) # null

Function with parameter with multiple return value

#Function with with parameter with multiple return value

Define plus()

```
def plus(a,b):  
    sum = a + b  
    return (sum, a,b)
```

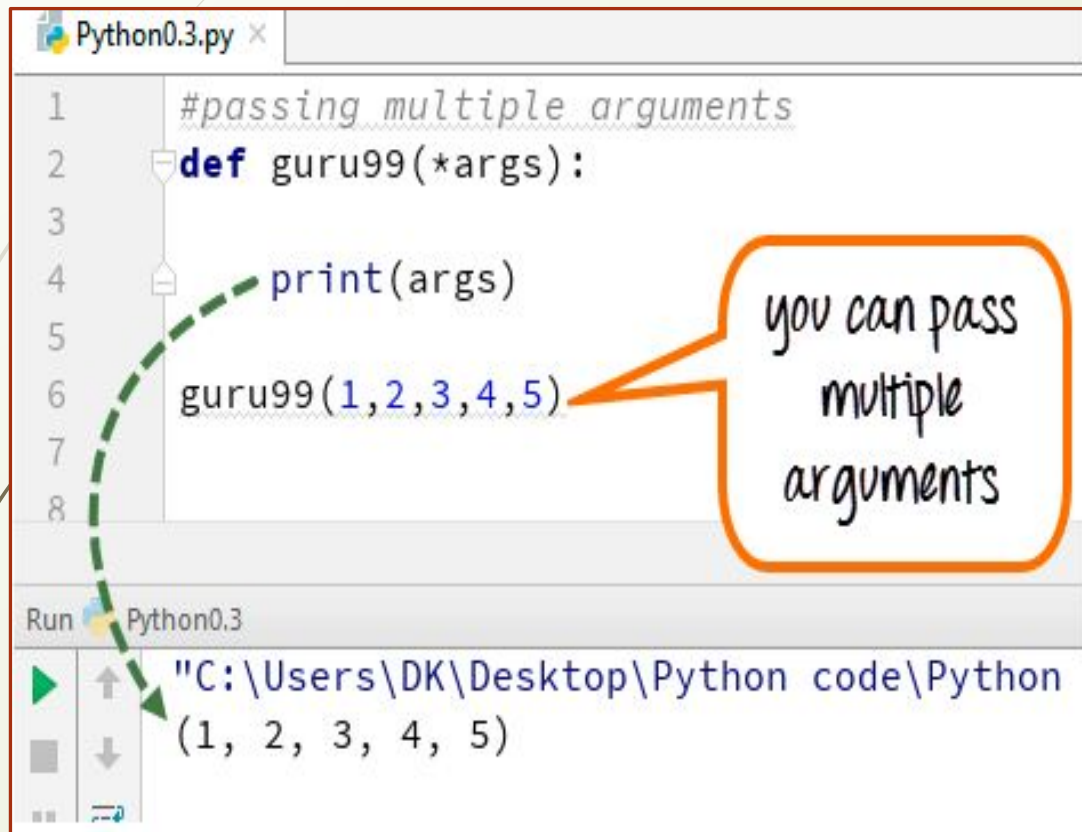
Call `plus()` and unpack variables

```
sum, a,b = plus(3,4)
```

Print `sum()`

```
print(a , "+" ,b , "=",sum)
```

Example on Function with parameter



```
Python0.3.py x
1  #passing multiple arguments
2  def guru99(*args):
3
4      print(args)
5
6      guru99(1,2,3,4,5)
7
8
```

you can pass multiple arguments

Run Python0.3

"C:\Users\DK\Desktop\Python code\Python (1, 2, 3, 4, 5)"

```
# Python code to demonstrate
# *args for dynamic arguments
def fn(*argList):
    for argx in argList:
        print (argx)

fn('I', 'am', 'Learning', 'Python')
```

Example on Function

Given an integer number **n**, define a function named **printDict()** which can print a dictionary where the keys are numbers between **1** and **n** (both included) and the values are square of keys.

The function **printDict()** doesn't take any argument.

```
def printDict():  
    n=int(input())  
    d={}  
    for i in range(1,n+1):  
        d[i]=i*i  
    print(d)  
printDict()
```